



UNIVERSIDAD VERACRUZANA

FACULTAD DE ESTADÍSTICA E INFORMÁTICA

XALAPA-ENRÍQUEZ, VERACRUZ

PROGRAMA EDUCATIVO

**LICENCIATURA EN INGENIERÍA EN SISTEMAS Y TECNOLOGÍAS DE LA
INFORMACIÓN**

EXPERIENCIA EDUCATIVA

TECNOLOGÍA PARA LA INTEGRACIÓN DE SOLUCIONES

DOCENTE

GÓMEZ ROMERO RAMÓN

TRABAJO

DOCUMENTO TÉCNICO DEL PROYECTO FINAL

INTEGRANTES DEL EQUIPO

BELLIDO MORA ADOLFO

BRUNO OLMOS ROBERTO ALFREDO

MORALES CARRIÓN JUAN JOSÉ

SUANEZ LANDA EPXON

17/06/2026



Sistema de Control de Acceso de Estacionamiento SICAE

Índice de contenido

Introducción	7
Objetivos	8
Objetivo General	8
Objetivos Específicos	8
Explicación de microservicios	9
Autenticación (AuthService)	9
Usuario (UserService)	11
Vehículo (VehicleService)	14
Estacionamiento (ParkingService)	19
Arquitectura del Sistema	21
Arquitectura de Microservicios	21
Arquitectura por Capas	22
Diagramas	23
Diagrama General de Arquitectura	23
Diagrama de Despliegue	24
Diagrama de componentes de una arquitectura en capas	25
Diagrama de Flujo de Autenticación JWT	26
Diseño de API REST	27
Tecnologías	31
Tecnologías de Desarrollo	31
Tecnologías de almacenamiento	31
Tecnologías de Despliegue	32
Pruebas	32
Pruebas de APIs	32
AUTHSERVICE	32
USERSERVICE	33
VEHICLESERVICE	33
PARKINGSERVICE	33
Casos exitosos	34
AUTHSERVICE	34
USERSERVICE	35



VEHICLESERVICE	37
PARKINGSERVICE	39
Casos de error	40
AUTHSERVICE	40
USERSERVICE	41
VEHICLESERVICE	48
PARKINGSERVICE	50
Evidencias	52
AUTHSERVICE	52
USERSERVICE	54
VEHICLESERVICE	69
PARKINGSERVICE	74
Conclusiones	81
Resultados obtenidos	81
Aprendizajes	82
Problemas encontrados	83



Índice de ilustraciones

Ilustración 1 Diagrama general de arquitectura	23
Ilustración 2 Diagrama de despliegue	24
Ilustración 3 Diagrama de componentes de una arquitectura en capas	25
Ilustración 4 Diagrama de flujo de autenticación con JWT	26
Ilustración 5 Evidencia de authService numero 1	52
Ilustración 6 Evidencia de authService numero 2	52
Ilustración 7 Evidencia de authService numero 3	53
Ilustración 8 Evidencia de userService numero 1	54
Ilustración 9 Evidencia de userService numero 2	54
Ilustración 10 Evidencia de userService numero 3	55
Ilustración 11 Evidencia de userService numero 4	55
Ilustración 12 Evidencia de userService numero 5	56
Ilustración 13 Evidencia de userService numero 6	56
Ilustración 14 Evidencia de userService numero 7	57
Ilustración 15 Evidencia de userService numero 8	57
Ilustración 16 Evidencia de userService numero 9	57
Ilustración 17 Evidencia de userService numero 10	58
Ilustración 18 Evidencia de userService numero 11	58
Ilustración 19 Evidencia de userService numero 12	58
Ilustración 20 Evidencia de userService numero 13	59
Ilustración 21 Evidencia de userService numero 14	59
Ilustración 22 Evidencia de userService numero 15	59
Ilustración 23 Evidencia de userService numero 16	60
Ilustración 24 Evidencia de userService numero 17	60
Ilustración 25 Evidencia de userService numero 18	60
Ilustración 26 Evidencia de userService numero 19	61
Ilustración 27 Evidencia de userService numero 20	61
Ilustración 28 Evidencia de userService numero 21	61
Ilustración 29 Evidencia de userService numero 22	62
Ilustración 30 Evidencia de userService numero 23	62
Ilustración 31 Evidencia de userService numero 24	62
Ilustración 32 Evidencia de userService numero 25	63
Ilustración 33 Evidencia de userService numero 26	63
Ilustración 34 Evidencia de userService numero 27	63
Ilustración 35 Evidencia de userService numero 28	64
Ilustración 36 Evidencia de userService numero 29	64
Ilustración 37 Evidencia de userService numero 30	64
Ilustración 38 Evidencia de userService numero 31	65
Ilustración 39 Evidencia de userService numero 32	65
Ilustración 40 Evidencia de userService numero 33	65
Ilustración 41 Evidencia de userService numero 34	66
Ilustración 42 Evidencia de userService numero 35	66
Ilustración 43 Evidencia de userService numero 36	66
Ilustración 44 Evidencia de userService numero 37	67



Ilustración 45 Evidencia de userService numero 38	67
Ilustración 46 Evidencia de userService numero 39	67
Ilustración 47 Evidencia de userService numero 40	68
Ilustración 48 Evidencia de userService numero 41	68
Ilustración 49 Evidencia de vehicleService numero 1	69
Ilustración 50 Evidencia de vehicleService numero 2	69
Ilustración 51 Evidencia de vehicleService numero 3	70
Ilustración 52 Evidencia de vehicleService numero 4	70
Ilustración 53 Evidencia de vehicleService numero 5	71
Ilustración 54 Evidencia de vehicleService numero 6	71
Ilustración 55 Evidencia de vehicleService numero 7	72
Ilustración 56 Evidencia de vehicleService numero 8	72
Ilustración 57 Evidencia de vehicleService numero 9	73
Ilustración 58 Evidencia de parkingService numero 1	74
Ilustración 59 Evidencia de parkingService numero 2	75
Ilustración 60 Evidencia de parkingService numero 3	76
Ilustración 61 Evidencia de parkingService numero 4	77
Ilustración 62 Evidencia de parkingService numero 5	78
Ilustración 63 Evidencia de parkingService numero 6	78
Ilustración 64 Evidencia de parkingService numero 7	79
Ilustración 65 Evidencia de parkingService numero 8	79
Ilustración 66 Evidencia de parkingService numero 9	80
Ilustración 67 Evidencia de parkingService numero 10	80



Índice de tablas

Tabla 1 Tabla sobre el diseño de API REST	27
---	----



Introducción

Los microservicios son un enfoque arquitectónico nativo de la nube en el que una sola aplicación se conforma de muchos componentes o servicios más pequeños, poco acoplados y que se pueden desplegar de forma independiente. Una arquitectura de microservicios es un tipo de arquitectura de aplicaciones en la que se desarrolla la aplicación como un conjunto de servicios. Este modelo proporciona el framework para desarrollar, desplegar y mantener aplicaciones y servicios de arquitectura de microservicios de forma independiente, pero ¿quién los usa?, ¿qué necesitan?, ¿cuándo ocuparlos?, ¿dónde nos benefician? y ¿por qué utilizarlos?

Quién, qué, cuándo, dónde y por qué son las preguntas que este proyecto final de la experiencia educativa de “Tecnologías para la Integración de Soluciones” nos ha ayudado a resolver a forma de cierre de esta materia. El presente documento muestra el trabajo de nuestro equipo ante una problemática: desarrollar un sistema distribuido de control de acceso para un estacionamiento privado basado en una arquitectura de microservicios orientada al dominio (Domain-Oriented Microservices), con separación funcional de servicios para la administración de usuarios, vehículos y registros de estacionamiento.

Se presentan los objetivos que buscamos cumplir, las características de cada uno de nuestros microservicios y la forma en que se relacionan entre sí para que el sistema pueda funcionar de manera ordenada. Para esto, se trabajó con servicios separados para autenticación, usuarios, vehículos y estacionamiento, manteniendo cada parte con responsabilidades propias y evitando que todo dependiera de un solo bloque de código.

También se explica la arquitectura general del sistema, la arquitectura por capas utilizada dentro de cada microservicio, los diagramas principales, el diseño de las APIs REST, las tecnologías utilizadas y las pruebas realizadas. Entre los elementos más importantes se encuentran el uso de JWT para la autenticación, bcrypt para el manejo seguro de contraseñas, bases de datos separadas por dominio y contenedores para facilitar el despliegue de los servicios.

Este proyecto nos permitió recordar y usar varios temas vistos durante el curso, como la separación de responsabilidades, la comunicación entre servicios, el uso de bases de datos independientes y la validación de reglas de negocio. Más allá de solo construir endpoints, el objetivo fue entender cómo se puede organizar un sistema distribuido para que cada servicio cumpla una función clara y pueda integrarse con los demás de forma sencilla.



Objetivos

Objetivo General

Construir un sistema funcional para el control de acceso a un estacionamiento privado, aplicando desde cero una arquitectura de microservicios. El propósito principal es poner en práctica lo aprendido sobre separación de responsabilidades, usar múltiples bases de datos independientes y lograr desplegar todo el proyecto de forma distribuida en máquinas virtuales Linux utilizando contenedores.

Objetivos Específicos

- Aplicar los fundamentos de microservicios para crear módulos (Usuarios, Vehículos, Parking y Auth) que trabajen de forma independiente y que no se rompan si otro microservicio falla.
- Organizar el código interno de cada microservicio mediante una arquitectura por capas (Controller, Service, Repository) para mantener todo ordenado y fácil de mantener a futuro.
- Lograr una persistencia de datos verdaderamente distribuida, asegurando que cada servicio administre exclusivamente su propia base de datos (PostgreSQL o MySQL) sin compartir tablas indebidamente.
- Proteger el acceso al sistema implementando un inicio de sesión seguro con JSON Web Tokens (JWT) y contraseñas encriptadas con Bcrypt.
- Aprender a utilizar Docker para "contenerizar" y levantar tanto las bases de datos como las aplicaciones de Java, simulando un entorno real de servidores.
- Validar y comprobar que todas las peticiones (APIs) funcionen correctamente haciendo pruebas detalladas y de casos de error con la herramienta Postman.



Explicación de microservicios

Autenticación (AuthService)

El servicio de autenticación es el primer microservicio de nuestro proyecto. AuthService tiene como finalidad realizar todas las acciones referentes a la seguridad y control de acceso. Con este microservicio podemos verificar la identidad de los usuarios y generar los tokens JWT necesarios para consumir el resto del sistema. De esta manera, separamos la lógica de validación de credenciales de los demás servicios, cumpliendo así con el propósito de trabajar con una arquitectura de microservicios.

Este microservicio se conecta de forma compartida a la base de datos de PostgreSQL `sicaeUsuario`, utilizando la misma tabla de usuarios que UserService para validar las credenciales.

Este microservicio cuenta con 1 funcionalidad principal establecida como requerimiento:

- Iniciar sesión: POST /auth/login

Comencemos desde el principio. Desde Postman se manda una petición a `http://localhost:8081/auth/login` utilizando el método POST. A diferencia del resto de los microservicios, esta es la única petición pública que no necesita un token proporcionado previamente en el header, ya que precisamente su objetivo es generar el primer token de la sesión.

Función login

La función login empieza cuando se manda una petición desde Postman al endpoint POST /auth/login. Esta petición debe traer en el cuerpo (body) un objeto JSON con los datos de acceso del usuario que quiere ingresar: username y password.

Primero, la petición llega al controller de nuestro microservicio. Ahí se identifica que la ruta corresponde al inicio de sesión. El cuerpo de la petición es capturado automáticamente mediante un objeto LoginRequestDTO. Posteriormente, este objeto, que contiene el usuario y la contraseña tal como se escribieron, se manda al método login de la clase service, dentro de un bloque try-catch para atrapar errores.

Una vez dentro del service, comienzan las validaciones de negocio. Primero, se utiliza el repository (UsuarioMapper) para ejecutar una consulta a la base de datos y buscar al usuario basándose en el username proporcionado. Si la consulta no regresa ningún resultado, significa que el usuario no existe en la base de datos, por lo que el sistema no permite continuar y devuelve un mensaje de error indicando que el usuario no existe.

Si el usuario sí existe en la base de datos, se revisa su estatus. Si el usuario está inactivo (baja lógica), el sistema tampoco permite continuar y devuelve un mensaje indicando que el usuario se encuentra inactivo.

Después de validar la existencia y el estatus, se realiza la validación de seguridad. Se obtiene la contraseña cifrada que está almacenada en la base de datos y se utiliza la herramienta BCrypt.checkpw para compararla de forma segura con la contraseña plana que ingresó el usuario desde Postman. Si BCrypt determina que no coinciden, se detiene la operación y se lanza un error indicando que la contraseña es incorrecta.



Si la validación de BCrypt es correcta, se procede a otorgar el acceso. El service llama a la clase JwtUtil, pasándole los datos del usuario. Esta clase se encarga de crear el JSON Web Token (JWT), firmarlo con nuestra clave secreta y asignarle un tiempo de expiración.

Cuando el token ya está generado, se empieza a construir el paquete de entrega. Los datos del usuario y el token recién creado se pasan a un objeto de tipo LoginResponseDTO. A este objeto se le asignan los datos básicos requeridos: el idUsuario, idRol, idTipoUsuario, username, el nombre completo (uniendo nombre y apellido paterno), sus roles mapeados en texto y el JWT.

Finalmente, el service devuelve este objeto DTO armado al controller. El controller lo recibe y responde a Postman con un ResponseEntity.ok, en el cual va un mensaje HTTP 200 positivo junto con el paquete JSON completo. A partir de este momento, Postman ya tiene el token y puede mostrar el mensaje indicando que el inicio de sesión fue exitoso. Si en alguna de las validaciones dentro del service algo falló, el error regresa al controller y este responde con un ResponseEntity.badRequest, mandando un HTTP 400 y el mensaje específico del error.



Usuario (UserService)

El servicio de usuario es el segundo microservicio de nuestro proyecto. UserService tiene como finalidad realizar todas las acciones que se pueden hacer dentro del sistema referentes a los usuarios. Con este microservicio podemos registrar usuarios, editarlos, ver su perfil y cambiar su estatus. De esta manera, separamos toda la lógica de los usuarios del resto del sistema, cumpliendo así con el propósito de trabajar con una arquitectura de microservicios.

Este microservicio cuenta con 4 funcionalidades principales establecidas como requerimientos:

- Agregar usuarios: POST /usuarios/agregar
- Editar usuarios: PUT /usuarios/editar/{idUserio}
- Ver perfil de usuario: GET /usuarios/verPerfil/{idUserio}
- Cambiar estatus de usuario: PATCH /usuarios/actualizarEstatus/{idUserio}/{idRol}

Comencemos desde el principio. Desde Postman se manda una petición a <http://localhost:8082/usuarios/> y, según el caso, se usa el método correspondiente. Para agregar se usa POST, para editar se usa PUT, para cambiar el estatus se usa PATCH y para ver el perfil se usa GET.

Todos los métodos necesitan un token proporcionado previamente por AuthService. Este token debe mandarse en la autorización de tipo Bearer, dentro del header de la petición. Después, este token llega al endpoint correspondiente, dependiendo de la acción que se quiera realizar.

Cada token que llega se verifica para saber si es válido, si fue creado con la misma clave que usa AuthService y si todavía no ha caducado. Si el token no cumple con alguna de estas reglas, el sistema no permite continuar con la operación y se devuelve un mensaje indicando que el token es inválido o ya expiró.

Función agregarUsuario

La función agregarUsuario empieza cuando se manda una petición desde Postman al endpoint POST /usuarios/agregar. Esta petición debe traer el token en el header y el cuerpo de la petición con los datos del nuevo usuario que se quiere registrar.

Primero, la petición llega al controller de nuestro microservicio. Ahí se identifica que la ruta corresponde a agregar usuario y, antes de pasar a la clase service, se revisa si la cabecera trae el token de autorización. Si no viene el token o no viene con el formato correcto de Bearer, se regresa un mensaje indicando que falta el token o que el formato es incorrecto.

Si el token sí viene bien, se le quita la palabra Bearer para dejar solamente el token limpio y después, ese token se manda al método correspondiente de la clase service, junto con el cuerpo de la petición. El cuerpo llega como un objeto UsuariosRequestDTO, que contiene la información que se quiere usar para crear al nuevo usuario.

Una vez dentro del service, primero se valida el token. Se revisa si el token es válido, si no está expirado y si fue creado con la misma clave que usa AuthService. Si el token no es correcto, se detiene el proceso y se devuelve un mensaje de error.

Después de validar el token, se obtiene el username que viene dentro del token. Con ese username se busca al usuario en la base de datos para saber quién está intentando hacer la operación porque



solo un administrador puede hacerlo. Si el usuario del token no existe en la base de datos o no tiene rol de administrador, el sistema no permite continuar.

Después de eso, se empieza a trabajar con el cuerpo de la petición. Los datos que llegaron en el UsuariosRequestDTO se pasan a un objeto de tipo Usuario, que representa la estructura que se maneja en la base de datos.

Luego se realizan las validaciones de los datos obligatorios. Se revisa que venga el rol, el tipo de usuario, el nombre, el apellido paterno, el programa educativo, el username, la contraseña, el correo y el teléfono. También se valida que el correo tenga un formato correcto.

Aunque en los requerimientos se menciona la clave de usuario y el tiempo de registro, estos datos no se mandan directamente desde Postman. En este microservicio se generan dentro del service. La clave de usuario se crea de forma interna siguiendo un patrón, y el tiempo de creación se asigna con la fecha y hora actual.

Después se revisa que no existan usuarios repetidos. Primero se busca si ya existe un usuario con el mismo username. Si ya existe, se regresa un mensaje diciendo que el nombre de usuario ya está en uso. Luego se busca si ya existe un usuario con el mismo correo. Si el correo ya está registrado, también se retorna un mensaje de error.

También se validan los catálogos para hacer uso del resto de tablas de nuestra base de datos, el sistema revisa si el id del programa educativo, el id del rol y el id del tipo de usuario existen realmente en sus tablas correspondientes y si alguno no existe, se devuelve un mensaje de error para explicar cuál dato está mal.

Si todo está correcto, se cifra la contraseña con BCrypt antes de guardarla en la base de datos. Y después se genera la clave de usuario, se asigna el estatus activo por defecto y se coloca el tiempo de creación.

Cuando ya está construido el objeto completo, se manda al repository. Esta es la última capa del flujo, porque aquí es donde se ejecuta la consulta hacia PostgreSQL. En este caso, se llama al método registrarUsuario, que hace el INSERT en la tabla usuario.

Si la consulta se ejecuta bien, el repository termina su trabajo y el resultado regresa al service. Después, el service devuelve un mensaje de operación exitosa. Ese mensaje sube de nuevo al controller y el controller lo responde con un ResponseEntity.ok, en el cual va un mensaje de 200 positivo junto con el mensaje que nosotros pusimos. Finalmente, Postman muestra el mensaje indicando que el usuario se agregó correctamente.

Función editarUsuario

La función editarUsuario permite actualizar la información de un usuario que ya existe. Esta función se prueba desde Postman con el método PUT y la ruta `http://localhost:8082/usuarios/editar/{idUsuario}`. En la URL se manda el id del usuario que se quiere editar y en el body se mandan los nuevos datos.

Igual que en las demás funciones, primero la petición llega al controller. El controller revisa que venga el token y hace todas las validaciones como en el servicio anterior y después, el controller manda el token limpio, el id del usuario y el body de la petición al método editarUsuario del service.



Ya dentro del service, primero se valida que el idUsuario venga presente. Luego se valida el token y si hacen las mismas acciones que el método anterior si es correcto o incorrecto y también se obtiene el username que viene dentro del token y se busca en la base de datos para saber si el usuario que quiere editar está dentro del sistema.

Después se busca al usuario que se quiere editar usando el idUsuario que llegó en la URL. Si ese usuario no existe, se devuelve un mensaje diciendo que el usuario que se quiere editar no existe.

Cuando el usuario sí existe, se crea un nuevo objeto de tipo Usuario llamado usuarioEditado. A este objeto se le asignan los datos que sí se pueden modificar, como el rol, tipo de usuario, programa educativo, nombre, apellido paterno, apellido materno, correo y teléfono.

Hay campos que no se pueden editar directamente desde este endpoint. Estos campos son el username, la contraseña y la clave de usuario. Por eso, aunque alguien los mande desde Postman, el service no los toma como datos nuevos porque ya están definidos a tomarse del usuario a editar original.

También se conserva el estatus y el tiempo de creación del usuario. El único tiempo que sí cambia es el tiempo de actualización, porque al editar el registro se le asigna la fecha y hora actual.

Después se hacen las validaciones. Se revisa que vengan los datos obligatorios como rol, tipo de usuario, nombre, apellido paterno, programa educativo, correo y teléfono. También se valida el formato del correo, el tamaño del nombre, el tamaño de los apellidos, el tamaño del correo y el tamaño del teléfono.

Luego se revisa que el correo no esté repetido. Aquí hay una validación importante: si el correo ya existe, pero pertenece al mismo usuario que está siendo editando, no hay problema. Pero si el correo pertenece a otro usuario, entonces se detiene la operación y se avisa que el correo electrónico ya está registrado.

También se vuelven a validar los parámetros de idRol, idTipoUsuario e idprogramaEducativo.

Si todo sale bien, se llama al repository y se ejecuta el método editarUsuario. Este método hace el UPDATE en la tabla usuario, cambiando solamente los campos permitidos. Finalmente, el resultado regresa al service, el service devuelve un mensaje de éxito y el controller responde a Postman con un ResponseEntity.ok.

Función verPerfil

La función verPerfil permite consultar la información completa de un usuario. Esta función se manda desde postman con el método GET y la ruta `http://localhost:8082/usuarios/verPerfil/{idUsuario}`. En la URL se manda el id del usuario cuyo perfil se quiere mostrar.

Primero, la petición llega al controller. Como en los demás casos, se revisa si viene el token en el header y se realiza todo como en anteriores casos, se manda al service junto con el IdUsuario y en el service, primero se valida que el idUsuario no venga vacío.

Se busca si el usuario existe o no, se manda un mensaje de error si no se encuentra o se continua con el flujo según el caso y se revisa el estatus del usuario. Si el usuario está inactivo, no se muestra su perfil. Esto se maneja como una baja lógica, por lo que el sistema devuelve un mensaje indicando que el usuario se encuentra inactivo.



Si el usuario existe y está activo, se empieza a construir un `UsuariosResponseDTO` con todos los datos de su perfil que queremos mostrar, no todos porque algunos no son necesarios

Una vez armado el DTO, el service lo regresa al controller. Luego, el controller responde con un `ResponseEntity.ok`, mandando el perfil completo a Postman. Si algo falla durante el proceso, se devuelve un mensaje indicando la causa por la que no se pudo completar la operación.

Función actualizarEstatus

La función `actualizarEstatus` sirve para cambiar el estatus de un usuario, ya sea de activo a inactivo o de inactivo a activo. Esta función se manda desde postman con el método `PATCH` y la ruta `http://localhost:8082/usuarios/actualizarEstatus/{idUsuario}/{idRol}`.

En esta ruta se mandan dos datos por la URL que son el `idUsuario` del usuario al que se le quiere cambiar el estatus y el `idRol` de ese mismo usuario. Además, como en las demás funciones, se debe mandar el token en el header con autorización `Bearer`.

Primero, la petición llega al controller. El controller revisa si viene el token hace la validación como siempre, si todo bien con el token, se limpia y manda al service como siempre junto con el `idUsuario` y el `idRol`.

Dentro del service, primero se valida que venga el `idUsuario`. Después se valida que venga el `idRol`. Si alguno de estos datos falta, se detiene la operación y se devuelve un mensaje indicando cuál dato falta, luego se valida el token y se hace lo de siempre si es válido, inválido o ya expiró y se extrae el username del usuario para saber si rol, si el usuario no es administrador el sistema no permite cambiar el estatus y devuelve el mensaje de siempre

Después se validan tanto el `idUsuario` como el `idRol`, se busca al usuario al que se le quiere cambiar el estatus usando el `idUsuario` que llegó en la URL y se valida que el `idRol` coincida.

También se añadió una condición extra que evita que el administrador se pueda cambiar el estatus a sí mismo, para evitar errores.

Si todas las validaciones se cumplen, se revisa el estatus actual del usuario. Si el usuario está activo, se cambia a inactivo. Si el usuario está inactivo, se cambia a activo. También se asigna el tiempo de actualización con la fecha y hora actual, porque el usuario se modificó en algo, en este caso el estatus.

Después se llama al repository y se ejecuta el método `actualizarEstatus`, que hace el `UPDATE` en la tabla usuario. Si todo sale bien, el resultado regresa al service, el service devuelve un mensaje de éxito y el controller responde a Postman con un `ResponseEntity.ok`.

Finalmente, Postman muestra el mensaje indicando que el estatus del usuario se actualizó correctamente. Si algo falla, se muestra un mensaje explicando por qué no se pudo completar la operación.

Vehículo (VehicleService)

El servicio de vehículos es el tercer microservicio de nuestro proyecto. `VehicleService` tiene como finalidad realizar todas las acciones relacionadas con los vehículos registrados dentro del sistema. Con este microservicio podemos registrar vehículos, consultar los vehículos de un usuario, editarlos, cambiar su estatus y validar si un vehículo puede ser usado por `ParkingService`. De esta manera,



separamos toda la lógica de vehículos del resto del sistema, cumpliendo con la arquitectura de microservicios.

Este microservicio trabaja con una base de datos independiente en MySQL llamada sicaevehiculo. En esta base se manejan las tablas de marca, modelo y vehiculo. Las marcas y modelos funcionan como catálogos, mientras que la tabla vehiculo guarda los vehículos asociados a los usuarios. El idUsuario se guarda en la tabla de vehículos, pero no tiene relación física con la base de usuarios porque ese dato pertenece a otro microservicio.

Este microservicio cuenta con 5 funcionalidades principales:

- Buscar vehículos por usuario: GET /vehiculos/buscarPorUsuario/{idUsuario}
- Agregar vehículo: POST /vehiculos/agregar
- Editar vehículo: PUT /vehiculos/editar/{idVehiculo}
- Cambiar estatus de vehículo: PATCH /vehiculos/actualizarEstatus/{idVehiculo}/{idUsuario}
- Validar vehículo para ParkingService: GET /vehiculos/validar?idUsuario={idUsuario}&placa={placa}

Desde Postman se manda una petición a <http://localhost:8083/vehiculos/> y, según el caso, se usa el método correspondiente. Para registrar se usa POST, para editar se usa PUT, para cambiar el estatus se usa PATCH y para consultar o validar se usa GET.

Todos los métodos principales necesitan un token proporcionado previamente por AuthService. Este token se manda en la autorización de tipo Bearer, dentro del header de la petición. Cuando el token llega al controller, primero se revisa que venga con el formato correcto. Después se le quita la palabra Bearer para dejar el token limpio y se manda al service.

Dentro del service, el token se valida para revisar si fue creado con la misma clave que usa AuthService y si todavía es válido. Si el token no cumple, el sistema no permite continuar y devuelve un mensaje indicando que el token es inválido.

Función buscarVehiculosPorUsuario

La función buscarVehiculosPorUsuario permite consultar todos los vehículos que pertenecen a un usuario específico. Esta función se prueba desde Postman con el método GET y la ruta <http://localhost:8083/vehiculos/buscarPorUsuario/{idUsuario}>.

Primero, la petición llega al controller. Ahí se revisa que venga el token de autorización. Si el token no viene o no tiene el formato Bearer, se devuelve un mensaje de error. Si el token viene correctamente, se limpia y se manda al service junto con el idUsuario.

Dentro del service se valida que el idUsuario no venga vacío y después se valida el token. Si todo está correcto, se llama al repository para buscar los vehículos del usuario. Para esta consulta se usa la vista vehiculofullinfo, porque ahí ya vienen juntos los datos del vehículo, modelo y marca.

El repository devuelve una lista de vehículos encontrados. Después, en el service, esos datos se pasan a objetos VehiculoResponseDTO para regresar una respuesta más ordenada a Postman. Finalmente, el controller responde con ResponseEntity.ok y muestra la lista de vehículos del usuario.

Función registrarVehiculo



La función registrarVehiculo permite agregar un nuevo vehículo al sistema. Esta función se manda desde Postman con el método POST y la ruta `http://localhost:8083/vehiculos/agregar`. La petición debe traer el token en el header y el body con los datos del vehículo.

El body incluye datos como idUsuario, idModelo, placa, color, año y descripción. El idVehiculo no se manda porque lo genera MySQL automáticamente. La claveVehiculo tampoco se manda desde Postman, porque se genera dentro del service siguiendo un patrón sencillo.

Primero, el controller recibe la petición y revisa que venga el token Bearer. Si viene bien, se limpia el token y se manda al service junto con el objeto VehiculoRequestDTO.

Dentro del service, primero se valida el token. Después se validan los datos obligatorios: idUsuario, idModelo, placa, color y año. También se revisan los tamaños de los campos para evitar errores con la base de datos, por ejemplo, que la placa no pase de 7 caracteres, el color no pase de 20 y la descripción no pase de 255.

Después se valida que el modelo exista y esté activo en el catálogo. También se busca si ya existe otro vehículo con la misma placa. Si la placa ya está registrada, se devuelve un mensaje de error porque no se permiten placas repetidas.

Luego se cuenta cuántos vehículos activos tiene el usuario. Si el usuario ya tiene 4 vehículos activos, el sistema no permite registrar otro. Esta validación se hizo porque el requerimiento indica que un usuario solo puede tener máximo 4 vehículos activos.

Si todo está correcto, se crea un objeto Vehiculo con los datos recibidos. Después se genera la claveVehiculo de forma interna, se asignan los datos al objeto y se manda al repository. El repository ejecuta el INSERT en la tabla vehiculo. Si todo sale bien, el service regresa un mensaje de éxito y el controller responde a Postman indicando que el vehículo se registró correctamente.

Función editarVehiculo

La función editarVehiculo permite modificar la información de un vehículo ya existente. Esta función se prueba con el método PUT y la ruta `http://localhost:8083/vehiculos/editar/{idVehiculo}`. En la URL se manda el idVehiculo y en el body se mandan los datos nuevos.

Primero, el controller revisa que venga el token de autorización. Si el token viene bien, se limpia y se manda al service junto con el idVehiculo y el body de la petición.

Dentro del service se valida que el idVehiculo venga presente y que el token sea válido. Después se validan los datos obligatorios del body, igual que en el registro: idUsuario, idModelo, placa, color y año. También se revisan los tamaños de placa, color y descripción.

Después se busca el vehículo en la base de datos usando el idVehiculo. Si no existe, se devuelve un mensaje de error. Si sí existe, se revisa que el vehículo pertenezca al idUsuario que llegó en la petición. Esto se hace para evitar que se edite un vehículo que no corresponde a ese usuario.

También se valida que el modelo nuevo exista y esté activo. Luego se revisa que la placa nueva no esté usada por otro vehículo. Esta validación ignora el vehículo actual, para que pueda conservar su misma placa sin marcarla como repetida.



Si todas las validaciones pasan, se crea un objeto Vehículo con los datos editables. En esta función no se cambia el idUsuario, la claveVehículo ni el estatus, porque esos datos no deben modificarse desde este endpoint. Finalmente, se llama al repository para hacer el UPDATE en la tabla vehículo. Si todo sale bien, el service devuelve un mensaje de éxito y el controller responde a Postman.

Función cambiarEstatus

La función cambiarEstatus sirve para activar o inactivar un vehículo sin eliminarlo físicamente. Esta función se manda desde Postman con el método PATCH y la ruta `http://localhost:8083/vehiculos/actualizarEstatus/{idVehículo}/{idUsuario}`.

En la URL se manda el idVehículo del vehículo que se quiere modificar y el idUsuario al que pertenece. Igual que en los demás métodos, también se manda el token Bearer en el header.

Primero, la petición llega al controller. El controller revisa el token y, si viene correctamente, lo manda al service junto con el idVehículo y el idUsuario.

Dentro del service se valida que vengan ambos datos. Después se valida el token. Luego se busca el vehículo por idVehículo. Si el vehículo no existe, se devuelve un mensaje de error. Si sí existe, se revisa que pertenezca al idUsuario enviado.

Si todo está correcto, se llama al repository para cambiar el estatus. La consulta hace que, si el vehículo está activo pase a inactivo, y si está inactivo vuelva a activo. Esto cumple con la regla de no eliminar físicamente los registros, sino manejarlos con baja lógica mediante el estatus.

Finalmente, si el cambio se realiza correctamente, el service devuelve un mensaje de éxito y el controller responde a Postman.

Función validarVehículoParaParking

La función validarVehículoParaParking es una de las más importantes para la integración con ParkingService. Su finalidad es permitir que el microservicio de estacionamiento pueda confirmar si una placa pertenece a un usuario y si el vehículo está activo.

Esta función se prueba con el método GET y la ruta `http://localhost:8083/vehiculos/validar?idUsuario={idUsuario}&placa={placa}`.

Primero, el controller recibe la petición y revisa que venga el token Bearer. Si el token viene bien, se manda al service junto con el idUsuario y la placa.

Dentro del service se valida que el idUsuario no venga vacío y que la placa tampoco venga vacía. Después se valida el token. Si todo está correcto, se llama al repository para buscar el vehículo en la vista vehiculofullinfo usando idUsuario y placa.

La consulta también valida que el vehículo esté activo. Si no se encuentra ningún registro, puede significar que el vehículo no existe, que no pertenece al usuario indicado o que está inactivo. En cualquiera de esos casos se devuelve un mensaje de error.

Si el vehículo sí existe, pertenece al usuario y está activo, el service arma un VehículoResponseDTO con los datos completos del vehículo, incluyendo marca, modelo, placa, color, año, estatus y descripción. Finalmente, el controller responde con ResponseEntity.ok y devuelve esos datos a Postman o al microservicio que lo consuma.



Así ParkingService puede validar vehículos sin consultar directamente la base de datos de vehículos, manteniendo la separación entre microservicios.



Estacionamiento (ParkingService)

El servicio de estacionamiento es el cuarto microservicio de este proyecto. ParkingService tiene como objetivo actuar como el orquestador final del sistema y controlar el acceso físico de los vehículos. Con este microservicio, se pueden registrar entradas, calcular cargos cuando se registran salidas y verificar la disponibilidad de estacionamiento. Este microservicio es central para la lógica de ocupación y cobro. Separa su base de datos del resto del sistema y realiza una arquitectura de microservicios orientada al dominio. Este microservicio tiene 3 funcionalidades principales definidas:

- Registrar Entrada: POST /parking/entry.
- Registrar Salida y Cobro: POST /parking/exit/{plate}.
- Verificar espacios disponibles: GET /parking/spaces.

Comencemos desde el principio. Desde Postman, se envía una solicitud a `http://localhost:8085/parking/` y, dependiendo del caso, se utiliza el método correspondiente. POST se utiliza para registrar entrada y salida y GET se utiliza para verificar espacios vacíos. Todos los métodos requieren un token proporcionado por AuthService. Este token debe enviarse en la autorización al tipo Bearer en el encabezado de la solicitud. Luego, este token llega al endpoint correspondiente dependiendo de la acción a realizar.

Función registrarEntrada

La función registrarEntrada comienza cuando se envía una solicitud desde Postman al endpoint POST /parking/entrada. Esta solicitud debe recibir el token en el encabezado y el cuerpo de la solicitud (un ParkingRequestDTO) con los datos que se requiere el userId y la placa del vehículo que desea ingresar.

Primero, la solicitud se envía al controlador de nuestro microservicio. Si el encabezado lleva el token de autorización, entonces se verifica si el token falta o no está en el formato correcto de Bearer. Se envía un mensaje de regreso si el token falta o si el formato es incorrecto.

Si el token es correcto, se elimina la palabra Bearer, dejando el token limpio y se envía al método correspondiente de la clase de servicio y se pasan el userId, la placa y el token completo.

Una vez en el servicio, las reglas de negocio son estrictas. Primero, se verifica la disponibilidad: el sistema escanea la base de datos para ver si hay algún espacio libre allí. Si el estacionamiento está lleno, se detiene y se envía un mensaje de error. Luego se valida el límite del usuario: cuenta cuántos vehículos tiene ese usuario que ya están en el auto y si tienen 2, no se les permite ingresar otro. Luego se valida que la placa no tenga ya un registro abierto y que el auto aún no haya registrado una doble entrada para ingresar.

A continuación, se realiza una validación local. El ParkingService utiliza el token que recibió y sale a comunicarse con los demás. Primero pregunta a UserService si el usuario existe y está activo. Luego pregunta a VehicleService si el vehículo existe, está activo y si realmente pertenece a ese usuario. Si alguna de estas consultas externas falla (el token está vencido, el usuario no existe o el auto no es suyo) se rechaza la solicitud. Si todo está en orden, entonces el sistema obtiene las tarifas de la tabla de configuración. Luego crea un objeto Ticket y le asigna el usuario, la placa, el espacio disponible, la tarifa por hora y la hora exacta de entrada con la fecha y hora actuales.



Finalmente, enviamos este objeto al repositorio. Allí, agregamos INSERT en la tabla de tickets y UPDATE en la tabla de espacios para marcar ese espacio como ocupado. El resultado regresa al servicio, que lo empaqueta en un ParkingResponseDTO (para mostrar todo en un formato JSON limpio). Este paquete se envía al controlador, que responde a Postman con ResponseEntity.ok mostrando detalles del movimiento asignado.

Función registrarSalida

La función registrarSalida permite registrar cuando un auto sale del estacionamiento y cuánto debe pagar por el estacionamiento. Esta función se prueba desde Postman utilizando el método POST y la ruta <http://localhost:8085/parking/salida/{paca}>.

A diferencia de la entrada, aquí no se utiliza cuerpo, por lo que la placa del vehículo se envía directamente a la URL.

Nuevamente, al igual que en el caso anterior, la solicitud se envía al controlador y se verifica que el token Bearer exista. Luego, el controlador envía la placa al método registrarSalida del servicio. Ya está dentro del servicio, por lo que lo primero es mirar la base de datos para ver si hay un ticket abierto (con estado "adentro") que corresponda a la placa. Si el auto no se encuentra dentro, se devuelve un error diciendo que el vehículo no fue encontrado. Si el ticket está allí, entonces se toma la fecha y hora actuales como hora de salida. En este punto se calcula el tiempo transcurrido, usando la matemática de java. Se calcula la diferencia entre la hora de entrada y salida en minutos totales para la precisión del cobro, luego dividimos esos minutos para obtener las horas completas cobradas y usamos el módulo (el resto de la división) para obtener los minutos extra. Luego se recupera la configuración de la base de datos y se multiplica por la tarifa de las horas y minutos para obtener el costo total. Una vez que se realizan todos los cálculos, el objeto Ticket original se actualiza poniendo la hora de salida, el tiempo total, las horas cobradas y el costo total. Luego se llama al repositorio para hacer dos cosas: UPDATE en la tabla de tickets (para que termine en 0 para guardar los costos) y UPDATE en la tabla de espacios (para que comience a estar libre nuevamente). Si todo va bien, el servicio hace un ParkingResponseDTO con el desglose del cobro y los tiempos.

Función consultarEspacios

La función consultarEspacios se puede utilizar para saber qué espacios de estacionamiento están libres en este momento. La función se envía desde Postman con el método GET y la ruta <http://localhost:8085/parking/spaces>.

Primero, la solicitud se envía al controlador donde se valida el token del Bearer. Si todo es correcto, el controlador llama al método consultarEspacios del servicio.

La lógica es muy simple dentro del servicio. Se llama al repositorio para completar una consulta SELECT a la tabla de espacios en la base de datos, pidiéndole que solo devuelva los identificadores de los espacios que están desocupados (ocupado = 0). El repositorio devuelve esta información como una lista de números que aún se envía al controlador de manera ResponseEntity.ok. Luego, el controlador envía un ResponseEntity.ok a Postman que muestra al usuario un array en JSON con todos los espacios que están disponibles. Si algo falla durante la solicitud, el catch captura el error y devuelve el mensaje de que la consulta no se pudo completar.



Arquitectura del Sistema

Arquitectura de Microservicios

La arquitectura de microservicios que utilizamos en SICAE divide el sistema en varios servicios independientes, donde cada uno se encarga de una parte específica del dominio del sistema. En lugar de tener una sola aplicación grande que controle usuarios, vehículos, autenticación y estacionamiento, se separó la funcionalidad en microservicios para que cada uno tenga una responsabilidad clara.

El sistema está formado por cuatro microservicios principales: AuthService, UserService, VehicleService y ParkingService. AuthService se encarga de autenticar a los usuarios y generar tokens JWT, UserService administra la información de los usuarios del sistema, VehicleService administra los vehículos registrados por los usuarios, incluyendo su registro, edición, cambio de estatus y validación y ParkingService controla las entradas, salidas, cobros y disponibilidad del estacionamiento.

Cada microservicio trabaja de forma independiente y expone sus funcionalidades mediante una API REST. La comunicación entre el cliente, como Postman, y los servicios se realiza mediante peticiones HTTP usando métodos como GET, POST, PUT y PATCH. Además, los servicios protegidos requieren un token generado por AuthService para permitir el acceso a sus endpoints.

Algo importante de esta arquitectura es que cada dominio cuenta con su propia base de datos. AuthService y UserService utilizan la base de datos de usuarios en PostgreSQL, VehicleService utiliza una base de datos MySQL para vehículos y ParkingService utiliza otra base de datos MySQL para los registros del estacionamiento. De esta forma podemos mantener separada la información de cada uno y evita que un microservicio dependa directamente de las tablas de otro.

La integración entre microservicios se muestra principalmente en ParkingService, ya que para registrar una entrada necesita validar información externa. Primero valida que el usuario exista y esté activo consultando a UserService. Después valida que el vehículo exista, pertenezca al usuario y esté activo consultando a VehicleService. De esta manera, ParkingService no accede directamente a las bases de datos de usuarios o vehículos, sino que consume los endpoints que cada microservicio ofrece.

Para el despliegue, los microservicios se organizan con Docker y Docker Compose, cada servicio puede levantarse en su propio contenedor junto con la base de datos que necesita. Esto nos permite probar los servicios de manera separada y también permite simular una arquitectura distribuida, donde cada parte del sistema puede ejecutarse de forma independiente.



Arquitectura por Capas

Dentro de nuestro sistema, se utiliza la arquitectura de microservicios junto con la arquitectura por capas para cada uno de ellos. Cada uno de los microservicios, AuthService, UserService, VehicleService y ParkingService, utiliza una distribución basada en Controller, Service y Repository. Esto se hace con el fin de separar responsabilidades para tratar la llegada de la petición, aplicar la lógica de negocio y, finalmente, acceder a los datos en cada base de datos.

El flujo inicia desde el cliente Postman, el cual lanza una petición HTTP. Cabe aclarar que esta comunicación se realiza mediante APIs REST y que el intercambio de información se hace en formato JSON. Esta petición llega primero a la capa Controller.

En la capa Controller se definen los endpoints de cada microservicio y se reciben las solicitudes de tipo GET, POST, PUT o PATCH, según el caso. En esta capa no se maneja lógica pesada, pues solo se encarga de recibir y enrutar las peticiones al proceso correspondiente.

Una vez que en la capa anterior se reciben las peticiones con sus datos, como el token o los parámetros de ruta, se procede a la segunda capa: Service. Aquí se encuentra la lógica de negocio de cada microservicio. En esta capa se validan los datos recibidos, se revisa si el token proporcionado es válido, se comprueba si el usuario tiene permisos y se aplican reglas como evitar registros duplicados, validar estatus activos o revisar que un vehículo pertenezca a cierto usuario.

Finalmente, según el caso, se accede a la última capa: Repository. Esta capa se encarga de la comunicación con la base de datos, ya sea PostgreSQL o MySQL, según el microservicio. AuthService y UserService utilizan la base de datos de usuarios en PostgreSQL, mientras que VehicleService utiliza una base de datos MySQL para vehículos y ParkingService utiliza otra base de datos MySQL para estacionamiento.

Además, se añade una capa más relacionada con el despliegue, donde cada microservicio y sus bases de datos correspondientes cuentan con contenerización mediante Docker. Esto permite desplegar cada servicio de manera independiente junto con su base de datos. La distribución de cada servicio quedó de la siguiente manera: un servidor para autenticación y usuarios, otro para vehículos y otro para estacionamiento.

El flujo de la arquitectura funciona de la siguiente manera: el cliente realiza una petición HTTP al microservicio correspondiente, el Controller recibe la solicitud, el Service valida las reglas de negocio y el Repository se comunica con la base de datos. Después, la respuesta regresa por el mismo camino hasta llegar nuevamente al cliente.



Diagramas

Diagrama General de Arquitectura

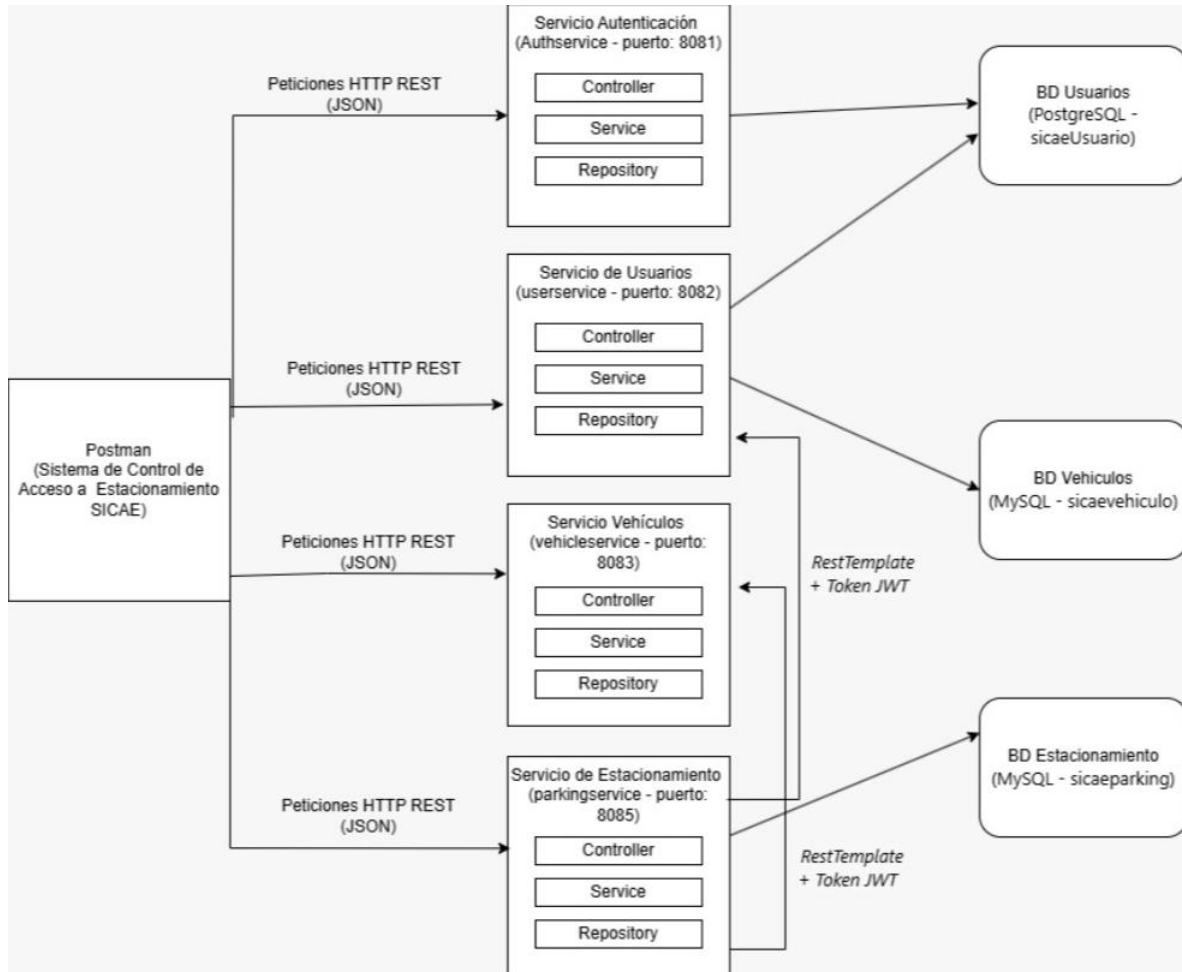


Ilustración 1 Diagrama general de arquitectura



Diagrama de Despliegue

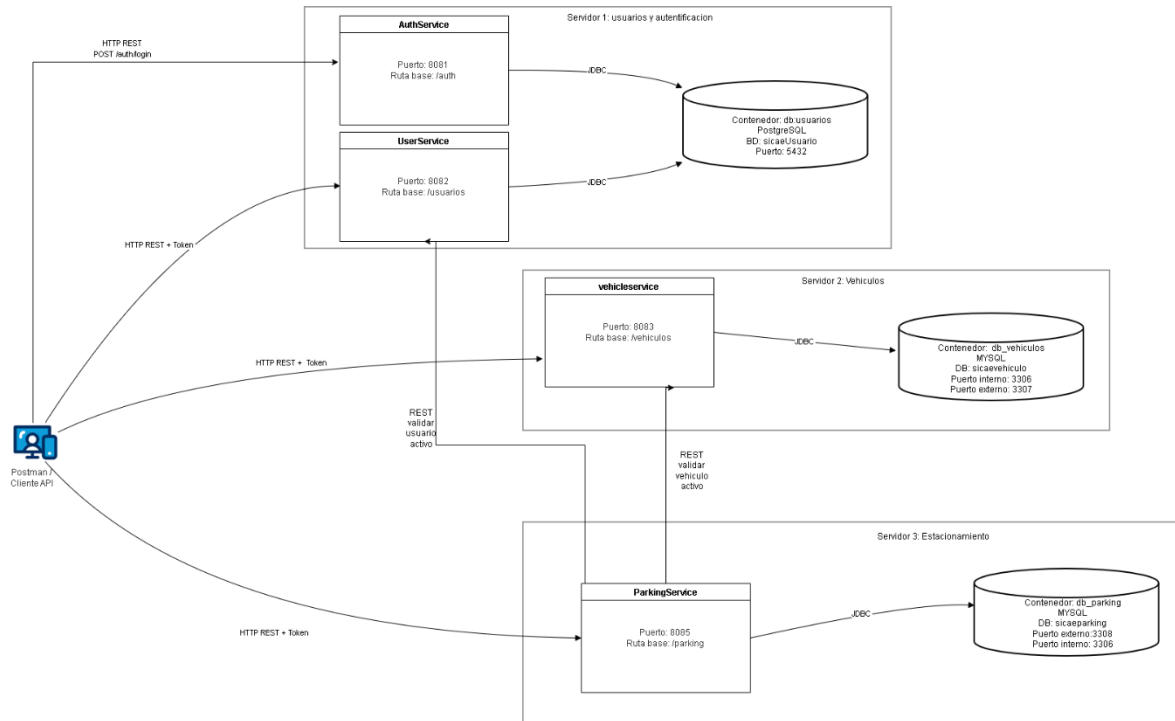


Ilustración 2 Diagrama de despliegue

Diagrama de componentes de una arquitectura en capas

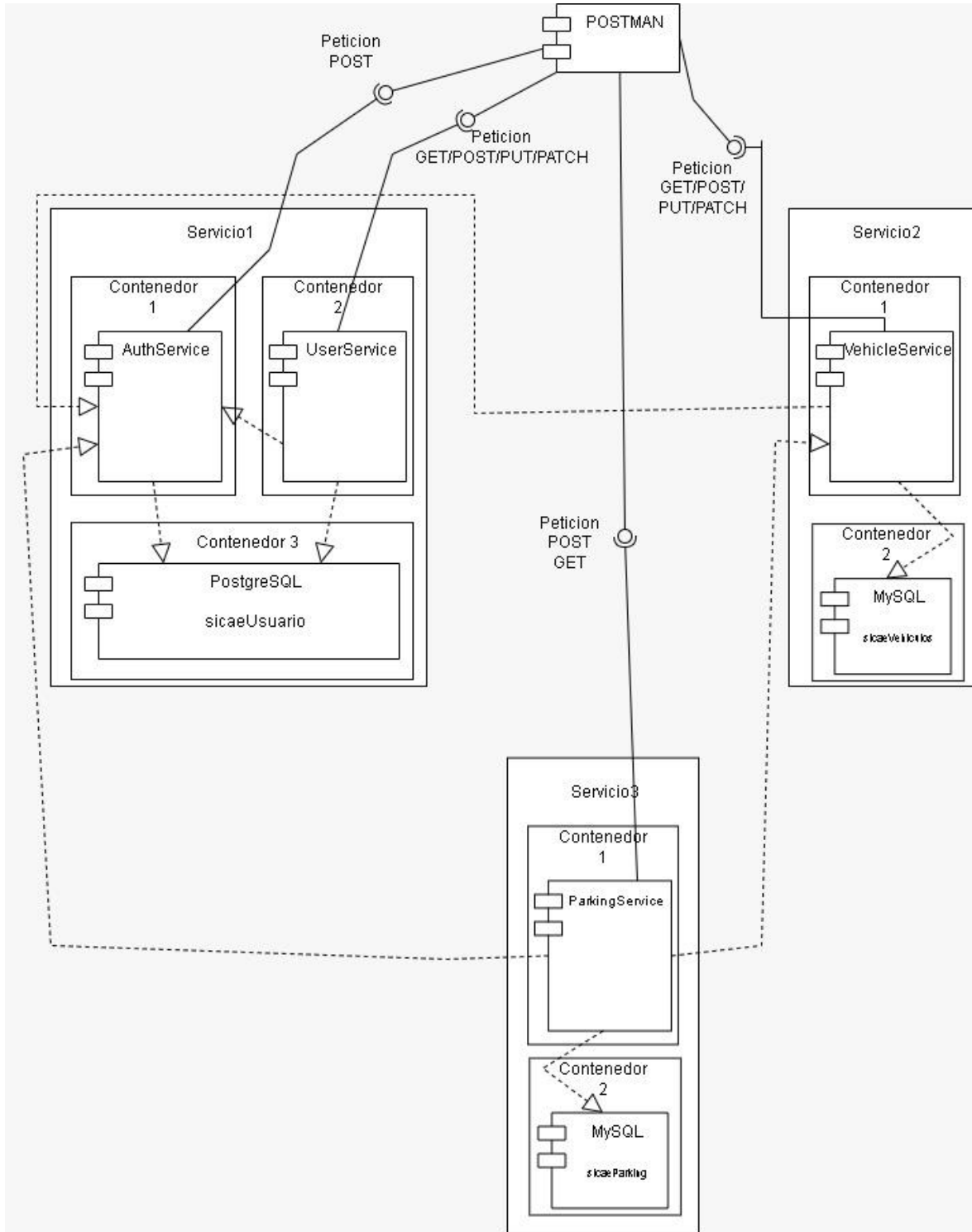


Ilustración 3 Diagrama de componentes de una arquitectura en capas



Diagrama de Flujo de Autenticación JWT

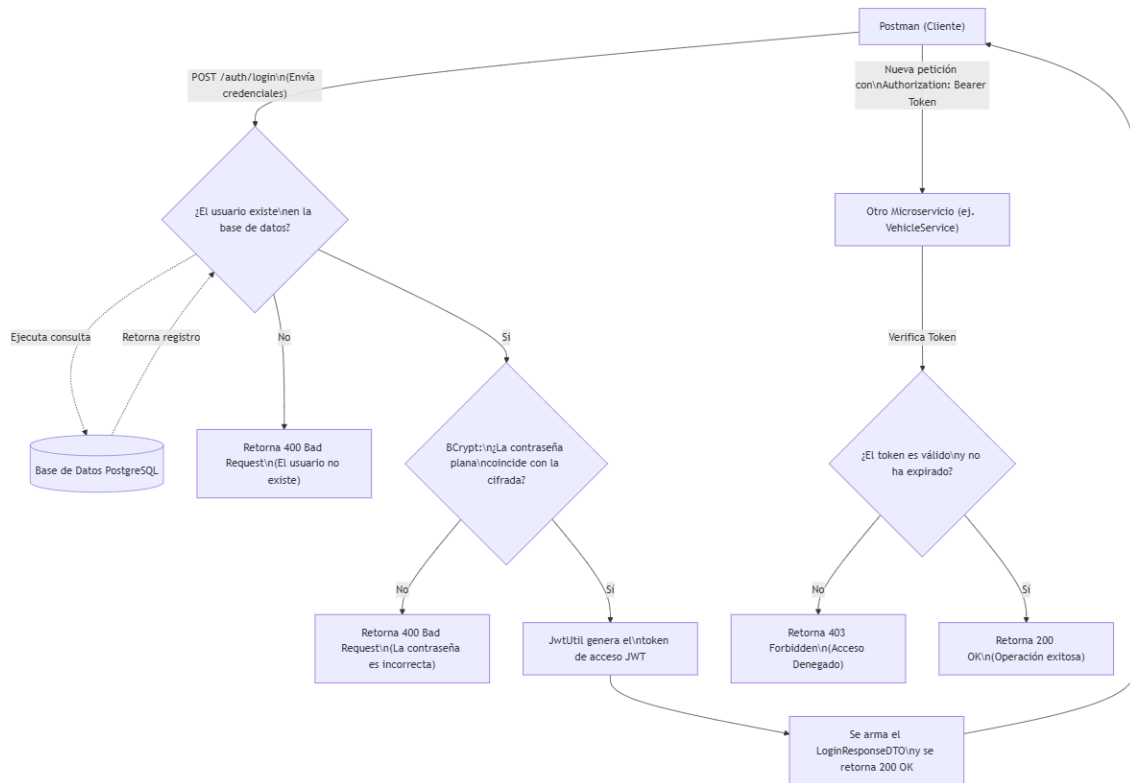


Ilustración 4 Diagrama de flujo de autenticación con JWT



Diseño de API REST

Tabla 1 Tabla sobre el diseño de API REST

Método	Endpoint	Descripción	Auth	Request	Response
POST	/auth/login	Permite iniciar sesión en el sistema. Valida las credenciales correctas y genera el JSON Web Token (JWT) necesario para autorizar las peticiones a los demás microservicios.	Sin token	{ "username": "admin", "password": "admin" }	200 OK { "idUsuario": 1, "idRol": 1, "rol": "ROL_ID_1", "usuario": "admin", "nombreCompleto": "Administrador fei", "idTipoUsuario": 1, "tipoUsuario": "TIPO_ID_1", "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzImlkVXN1YXJpbyI6MSwiaWRBb2wiOiJEsImhhdCI6MTc4MTY1NDMwNCwiZXhwIjoxNzgxNjYxNTA0fQ.IpoL5C3S2F5StjdL7Watnqwp8ssYMd_abnolC8c095E"
POST	/auth/login	Intento de inicio de sesión con usuario inexistente. Valida que el sistema rechace la petición cuando el username no está registrado en la base de datos PostgreSQL.	Sin token	{ "username": "123", "password": "admin" }	400 Bad Request El usuario no existe
POST	/auth/login	Intento de inicio de sesión con contraseña incorrecta. Valida que la herramienta Bcrypt bloquee el acceso si la contraseña ingresada no coincide con el hash almacenado.	Sin token	{ "username": "admin", "password": "123" }	400 Bad Request La contraseña es incorrecta
POST	/usuarios/agregar	Permite registrar un nuevo usuario en el sistema. Solo lo puede hacer un	Bearer Token	{ "idRol": 2, "idTipoUsuario": 1,	Operación realizada correctamente: El usuario se agregó con éxito



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

		administrador con token válido.		<pre>"idProgramaEducativo": 1, "nombre": "Johan", "apellidoPaterno": "Andrade", "apellidoMaterno": "Merino", "email": "johan@j2rp.com", "telefono": "2281234567", "username": "johandrade", "password":"123!" }</pre>	
PUT	/usuarios/editar/{idUserario}	Permite editar la información de un usuario existente. Se manda el id del usuario en la URL y los datos nuevos en el body		<pre>{ "idRol": 2, "idTipoUsuario": 1, "idProgramaEducativo": 1, "nombre": "Johan Editado", "apellidoPaterno": "Andrade", "apellidoMaterno": "Merino", "email": "johan.editado@j2rp.com", "telefono": "2281234567" }</pre>	Operación realizada correctamente: El usuario se editó con éxito
GET	/usuarios/verPerfil/{idUserario}	Permite consultar la información completa de un usuario usando su idUsuario.		Sin body, solo el IdUsuario en la URL Ejemplo: /usuarios/verPerfil/	<pre>{ "idUsuario": 4, "rol": 2, "nombre": "Carlos Alberto", "apellidoPaterno": "Ramirez", "apellidoMaterno": "Lopez", "tipoUsuario": 3, "programaEducativo": 5,</pre>



					<pre> "usuario": "Changolokoyosoy", "correo": "carlos.edicion@sicae.co m", "telefono": "2281112233", "estatus": true, "claveUsuario": "PRU- 125", "tiempoCreacion": "2026-06- 12T06:13:47.476322", "tiempoActualizacion": null } </pre>
PATCH	/usuarios/actualizarEstatus/{idUsuario}/{idRol}	Permite cambiar el estatus de un usuario de activo a inactivo o de inactivo a activo. Solo lo puede hacer un administrador.		Sin body, solo idUsuario e idRol en la URL	Operación realizada correctamente: El estatus del usuario se actualizó con éxito
GET	/vehiculos/buscarPorUsuario/{idUsuario}	Permite consultar todos los vehículos asociados a un usuario específico, necesita un token válido.	Bearer Token	No ocupa body pq en la URL se manda el idUsuario por ejemplo: /vehiculos/buscarPorUsuario/2	Lista de vehículos del usuario con idVehiculo, idUsuario, claveVehiculo, marca, modelo, placa, color, año, estatus y descripción.
POST	/vehiculos/agregar	Permite registrar un nuevo vehículo asociado a un usuario. Valida que la placa sea única, que el modelo exista y máximo 4 vehículos activos por usuario.	Bearer Token	{ "idUsuario": 2, "idModelo": 1, "placa": "ABC1234", "color": "Rojo", "anio": 2020, "descripcion": "vehiculo de prueba" }	El vehiculo se registró exitosamente
PUT	/vehiculos/editar/{idVehiculo}	Permite editar los datos de un vehículo existente. No se puede modificar el idUsuario, claveVehiculo ni estatus.	Bearer Token	json { "idUsuario": 2, "idModelo": 4, "placa": "XYZ9876", "color": "Azul", "anio": 2021, "descripcion": "vehiculo editado" }	El vehiculo se actualizo correctamente
PATCH		Permite cambiar el estatus de un vehículo de activo a inactivo o	Bearer Token	No ocupa body ya que en la URL se manda idVehiculo e	estatus del vehiculo cambiado con éxito!!



		de inactivo a activo, sin eliminarlo.		idUsuario. Ejemplo: /vehiculos/actualizarEstatus/1/2	
GET	/vehiculos/validar?idUsuario={idUsuario}&placa={placa}	Permite validar si una placa pertenece a un usuario y si el vehículo está activo. Este endpoint lo usa ParkingService.	Bearer Token	No ocupa body porque los datos van como parámetros: /vehiculos/validar?idUsuario=2&placa=XYZ9876	Datos completos del vehículo si existe, pertenece al usuario y está activo. Si no cumple devuelve mensaje de error.
POST	/parking/entrada	Registra la entrada de un vehículo al estacionamiento. Valida que haya espacios, que el usuario no exceda su límite y que el vehículo esté asignado a él.	Bearer Token	{ "idUsuario": 2, "placa": "ABC1234" }	{"idMovimiento": 1, "tiempoEntrada": "2026-06-15T10:00:00.000", "tiempoSalida": null, "espacioAsignado": 1, "tarifaHora": 15.0, "costoTotal": null, "horasCobradas": null}
POST	/parking/salida/{placa}	Registra la salida de un vehículo del estacionamiento. Libera el cajón asignado y calcula automáticamente el tiempo transcurrido y el costo total a pagar.	Bearer Token	No lleva body, pero si lleva la placa al final de la URL, por ejemplo: /parking/salida/ABC1234	{ "idMovimiento": 1, "tiempoEntrada": "2026-06-15T10:00:00.000", "tiempoSalida": "2026-06-15T12:30:00.000", "espacioAsignado": 1, "tarifaHora": 15.0, "costoTotal": 37.5, "horasCobradas": 2 }
GET	/parking/espacios	Consulta la lista de los cajones de estacionamiento que se encuentran disponibles (desocupados) en este momento.	Bearer Token	Sin body	[1,2,3,4,...,49, 50]



Tecnologías

Tecnologías de Desarrollo

El proyecto fue desarrollado en lenguaje Java, que fue el principal para todos los microservicios, también utilizamos Spring Boot para mantener una estructura parecida en todos los servicios y que fuera fácil la comunicación entre microservicios.

Los microservicios desarrollados fueron AuthService, UserService, VehicleService y ParkingService.

- AuthService se encarga de la generación de tokens validos e inicio de sesión de los usuarios.
- UserService se encarga de operaciones con los usuarios, como su información, estado y agregación.
- VehicleService Se encarga de los vehículos dentro del sistema
- ParkingService controla las entradas, salidas y espacios del estacionamiento.

Como framework principal usamos Spring Boot, ya que por conocimiento previo, se nos facilitó crear un sistema manera ordenada y fácil al trabajar con APIs REST. Dentro de cada microservicio se usaron controladores con `@RestController`, los cuales reciben las peticiones HTTP y las mandan a la capa correspondiente para aplicar la lógica de negocio.

También se utilizó una arquitectura por capas, principalmente dividida en Controller, Service y Repository. La capa Controller recibe las peticiones, la capa Service contiene la lógica de negocio y la capa Repository se encarga de comunicarse con la base de datos.

Para la construcción y manejo de dependencias se utilizó Maven. Cada microservicio cuenta con su propio archivo pom.xml, donde se declaran Spring Web, MyBatis, las conexiones a la base de datos, JWT, Bcrypt y aparte Maven Wrapper, para poder ejecutar comandos de Maven si hacer tanta configuración a cada equipo.

Para la persistencia de datos desde Java se utilizó MyBatis. Esta herramienta nos permite consultar a las bases de datos desde el repositorio y permitió tener mayor control sobre las consultas que hicimos a lo largo del proyecto.

En cuanto a seguridad, se utilizó JWT para la autenticación y autorización de las peticiones. AuthService genera el token cuando el usuario inicia sesión con los datos correctos, y los demás microservicios validan ese token antes de permitir ciertas operaciones. También se utilizó Bcrypt para cifrar las contraseñas antes de guardarlas en la base de datos para no guardarlas en texto plano

En ParkingService utilizamos RestTemplate para comunicarse con los otros microservicios. Esto se usa principalmente cuando se registra una entrada al estacionamiento, ya que ParkingService necesita validar que el usuario exista y esté activo en UserService, además de validar que el vehículo exista y pertenezca al usuario en VehicleService.

Como herramientas de desarrollo y pruebas usamos NetBeans y Postman. NetBeans fue el entorno de desarrollo usado para trabajar con los microservicios, mientras que Postman se usó para probar los endpoints, enviar tokens en las peticiones y validar los casos exitosos y de error.

Tecnologías de almacenamiento

Para almacenar los datos se utilizaron bases de datos separadas por dominio, donde cada dominio controla y accede a su propia base de datos según se requiera.



Para el dominio de usuarios se utilizó PostgreSQL. Esta base de datos se llama sicaeUsuario y contiene las tablas de usuarios, roles, tipos de usuario y programas educativos. Esta misma base es utilizada por AuthService para validar las credenciales del usuario y por UserService para registrar, editar, consultar y cambiar el estatus de los usuarios.

Para el dominio de vehículos se utilizó MySQL. Esta base de datos se llama sicaevehiculo y contiene las tablas de marca, modelo y vehículo, permitiendo registrar vehículos, editarlos, cambiar su estatus, consultarlos por usuario y validarlos cuando ParkingService lo necesita.

Para el dominio de estacionamiento también se utilizó MySQL, su base se llama sicaeparking. Esta base tiene la información de los cajones del estacionamiento, la configuración de costos y los tickets o movimientos de entrada y salida. De esta manera el servicio sabe qué espacios están disponibles, registrar entradas, registrar salidas y calcular el costo correspondiente.

Tecnologías de Despliegue

Para el despliegue de nuestro sistema se utilizó Docker. Cada microservicio cuenta con su propio Dockerfile, donde se define la imagen base de Java, la carpeta de trabajo, el archivo .jar que se ejecutará y el puerto.

También se utilizó Docker Compose para levantar varios contenedores al mismo tiempo. En el proyecto se dividió el despliegue en tres archivos principales, uno por servidor, siendo:

- Servidor1
- Servidor2
- Servidor3

En el Servidor 1 se despliegan AuthService, UserService y la base de datos PostgreSQL de usuarios. Los contenedores se conectan mediante la red interna de Docker y utilizan un volumen para conservar los datos de PostgreSQL aunque el contenedor se apague.

En el Servidor 2 se despliegan VehicleService y la base de datos MySQL de vehículos. Para vehicle Docker Compose levanta un contenedor para MySQL y otro para el microservicio de vehículos. Ambos se conectan dentro de la red propia para que el microservicio pueda comunicarse con su base de datos sin tener problemas con la externa.

En el Servidor 3 se despliegan ParkingService y la base de datos MySQL de estacionamiento. También se comunica con UserService y VehicleService para validar datos antes de permitir el acceso de algún vehículo al estacionamiento.

Para las imágenes de Java se utilizaron imágenes de Eclipse Temurin en versiones 17 y 21, otros de nuestros servicios se ejecutan con Java 17 y otros con Java 21 en sus Dockerfiles.

Pruebas

Pruebas de APIs

AUTHSERVICE

Para probar el funcionamiento del microservicio AuthService se utilizó Postman con los contenedores desplegados en Docker. Al ser el servicio central encargado de generar los accesos para el resto del proyecto, las pruebas consistieron en realizar peticiones de tipo POST (sin requerir



token previo) validando las credenciales contra la base de datos compartida de usuarios. Las pruebas confirmaron la generación exitosa del token JWT, así como la correcta aplicación de filtros de seguridad para bloquear credenciales inválidas.

USERSERVICE

Para probar que el microservicio de UserService funcionara correctamente, se utilizó Postman tanto en la fase de pruebas con NetBeans como en el despliegue final. Primero, se realiza el login en AuthService para obtener un token válido según el tipo de usuario. Con él, se puede acceder a las distintas funciones del microservicio, que utiliza peticiones de tipo GET, POST, PUT y PATCH para realizar las distintas operaciones a los usuarios. Es necesario que, según el servicio, lo solicite un usuario administrador o un usuario normal. De esta manera, se cumple con los requisitos de seguridad solicitados.

VEHICLESERVICE

Para probar el funcionamiento de VehicleService se utilizó Postman, primero se realizó una petición al microservicio AuthService para obtener un token JWT válido, después ese token se puso en las peticiones de VehicleService usando autorización de tipo Bearer Token.

PARKINGSERVICE

Para comprobar que la funcionalidad del microservicio ParkingService cumpliera con su funcionalidad se utilizó Postman con contenedores desplegados en Docker, después de obtener el token JWT desde AuthService, se obtuvo con éxito las pruebas de los endpoints para consultar espacios libres (en método GET), registrar entradas al estacionamiento verificando las reglas de negocio con otros servicios (método POST), y registrar la salida de los vehículos con el cálculo automático de costos (POST). Las Pruebas incluyeron validaciones de seguridad (tokens falsos), restricciones de negocio.



Casos exitosos

AUTHSERVICE

Login exitoso:

- Se valida que un usuario pueda iniciar sesión de manera exitosa, siempre y cuando proporcione sus credenciales correctas en el cuerpo de la petición.
- Datos de entrada:
 - Auth Type configurado como No Auth (al ser una petición pública).
 - Cuerpo JSON con: {"username": "admin", "password": "admin"}
- Resultado: El sistema busca al usuario en la base de datos, verifica mediante Bcrypt que la contraseña coincida y, una vez cubiertas las validaciones, genera el token de acceso JWT y devuelve:
 - {"idUsuario": 1, "idRol": 1, "rol": "ROL_ID_1", "usuario": "admin", "nombreCompleto": "Administrador fei", "idTipoUsuario": 1, "tipoUsuario": "TIPO_ID_1", "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJhZG1pbGlzImkVXN1YXJpbyI6MSwiaWRSc2wiOiJEsImldCI6MTc4MTY1NDMwNCwiZXhwIjoxNzg5NTA0fQ.IpoL5C3S2F5StjdL7Watnqwp8ssYMD_abnoIC8c095E"}



USERSERVICE

Agregar usuario:

- Se valida que el administrador pueda agregar un nuevo usuario, siempre y cuando cuente con un token valido, sea administrador y los parámetros en el cuerpo de body se cumplan
- Datos de entrada:
 - token obtenido de authservice del administrador y puesto en Auth Type Bearer.
 - Cuerpo JSON con: {"idRol": 2,"idTipoUsuario": 3,"idProgramaEducativo": 1,"nombre": "Prueba","apellidoPaterno": "Prueba","apellidoMaterno": "Registro","email": "juan.registro001@sicae.com","telefono": "2281234567","username": "juanregistro001","password": "Password123"}
- Resultado: El sistema analiza los parámetros obligatorios, como el token, IdUsuario, cuerpo JSON y los valida. Una vez cubiertas las validaciones, registra al usuario y devuelve:
 - “Operación realizada correctamente: El usuario se agregó con éxito”

Editar usuario:

- Se valida que un usuario pueda editar la información de un usuario existente, siempre y cuando cuente con un token válido, el idUsuario exista y los parámetros enviados en el cuerpo del body se cumplan. No se editan datos como usuario, contraseña y clave de usuario.
- Datos de entrada:
 - token obtenido de AuthService y puesto en Auth Type Bearer.
 - idUsuario enviado en la URL: 6
 - Cuerpo JSON con: {"idRol": 2,"idTipoUsuario": 1,"idProgramaEducativo": 1,"nombre": "Johan Editado","apellidoPaterno": "Andrade","apellidoMaterno": "Merino","email": "johan.editado@j2rp.com","telefono": "2281234567"}
- Resultado: El sistema analiza los parámetros obligatorios, como el token, idUsuario y cuerpo JSON. Después el sistema hace las validaciones correspondientes y una vez cubiertas, actualiza la información del usuario y devuelve:
 - “Operación realizada correctamente: El usuario se editó con éxito”

Ver perfil de usuario:

- Se valida que un usuario pueda ver la información completa de su perfil, siempre y cuando cuente con un token válido y proporcione correctamente el idUsuario en la URL.
- Datos de entrada:
 - token obtenido de AuthService y puesto en Auth Type Bearer.
 - idUsuario enviado en la URL: 3
- Resultado: El sistema analiza los parámetros obligatorios y una vez cubiertas las validaciones, devuelve los datos completos del usuario:
 - {"idUsuario": 3,"rol": 2,"nombre": "Monserrat","apellidoPaterno": "Hernandez","apellidoMaterno": "Niato","tipoUsuario": 3,"programaEducativo": 1,"usuario": "Monchis123","correo": "monchis@sicae.com","telefono": "2281234567","estatus": true,"claveUsuario": "PRU-124","tiempoCreacion": "2026-06-12T05:25:23.862914","tiempoActualizacion": null}

Cambiar estatus de usuario:



- Se valida que el administrador pueda cambiar el estatus de un usuario existente, siempre y cuando cuente con un token válido, sea administrador y proporcione correctamente el idUsuario y el idRol en la URL.
- Datos de entrada:
 - token obtenido de AuthService del administrador y puesto en Auth Type Bearer.
 - idUsuario enviado en la URL: 4
 - idRol enviado en la URL: 2
- Resultado: El sistema analiza los parámetros obligatorios y realiza validaciones internas, una vez cubiertas las validaciones, cambia el estatus del usuario y devuelve:
 - “Operación realizada correctamente: El estatus del usuario se actualizó con éxito”



VEHICLESERVICE

Login AuthService:

- Se valida que el usuario pueda iniciar sesión correctamente usando credenciales válidas. Esta prueba se realiza para obtener el token que después se utiliza en las peticiones de VehicleService.
- Datos de entrada:
 - Cuerpo JSON con: {"username": "admin1", "password": "contra1"}
- Resultado: El sistema valida las credenciales del usuario en AuthService y devuelve los datos del usuario junto con un token JWT válido.

Registrar vehículo:

- Se valida que se pueda registrar un nuevo vehículo asociado a un usuario, siempre que se mande un token válido y los parámetros del body cumplan con las reglas requeridas.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Cuerpo JSON con: {"idUsuario": 2, "idModelo": 1, "placa": "ABC1234", "color": "Rojo", "anio": 2020, "descripcion": "vehiculo de prueba"}
- Resultado: El sistema valida el token, los datos obligatorios, el modelo, la placa única y el límite de vehículos activos. Una vez cubiertas las validaciones, registra el vehículo y devuelve:
 - "El vehículo se registró exitosamente"

Buscar vehículos por usuario:

- Se valida que el sistema pueda consultar los vehículos asociados a un usuario específico, siempre que se mande un token válido.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - idUsuario enviado en la URL: /vehiculos/buscarPorUsuario/2
- Resultado: El sistema valida el token y el idUsuario. Después consulta los vehículos asociados al usuario y devuelve una lista con los datos completos de cada vehículo, incluyendo marca, modelo, placa, color, año, estatus y descripción.

Editar vehículo:

- Se valida que se pueda editar la información principal de un vehículo existente, siempre y cuando el vehículo exista, pertenezca al usuario indicado y se mande un token válido.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - idVehiculo enviado en la URL: /vehiculos/editar/1
 - Cuerpo json con: {"idUsuario": 2, "idModelo": 4, "placa": "XYZ9876", "color": "Azul", "anio": 2021, "descripcion": "vehiculo editado"}
- Resultado: El sistema valida el token, el idVehiculo, los datos obligatorios, que el vehículo pertenezca al usuario, que el modelo exista y que la placa no esté registrada en otro vehículo. Una vez cubiertas las validaciones, actualiza el vehículo y devuelve:
 - "El vehiculo se actualizo correctamente"



Cambiar estatus:

- Se valida que se pueda cambiar el estatus de un vehículo sin eliminarlo físicamente, siempre y cuando el vehículo exista, pertenezca al usuario indicado y se mande un token válido.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - idVehiculo e idUsuario enviados en la URL: /vehiculos/actualizarEstatus/1/2
- Resultado: El sistema valida el token, el idVehiculo y el idUsuario. Después verifica que el vehículo exista y pertenezca al usuario indicado. Una vez validado, cambia el estatus del vehículo de activo a inactivo o de inactivo a activo y devuelve:
 - “Estatus del vehiculo cambiado con éxito!!”

Validar vehículo para ParkingService:

- Se valida que el sistema pueda confirmar si un vehículo pertenece a un usuario y se encuentra activo. Esta prueba sirve para que ParkingService pueda usar este endpoint antes de registrar entradas o salidas.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - idUsuario y placa enviados como parámetros en la URL: /vehiculos/validar?idUsuario=2&placa=XYZ9876
- Resultado: El sistema valida el token, el idUsuario y la placa. Después consulta si existe un vehículo con esa placa, si pertenece al usuario indicado y si está activo. Cuando todo se cumple, devuelve los datos completos del vehículo.



PARKINGSERVICE

Login AuthService:

- Se valida que el usuario pueda iniciar sesión correctamente usando credenciales válidas. Esta prueba se realiza para obtener el token que después se utiliza en todas las peticiones de ParkingService.
- Datos de entrada:
 - Cuerpo JSON con: {"username": "admin1", "password": "contra1"}
- Resultado: El sistema valida las credenciales del usuario en AuthService y devuelve los datos del usuario junto con un token JWT válido.

Consultar espacios disponibles:

- Se valida que el sistema pueda consultar la lista de cajones de estacionamiento que se encuentran libres, siempre que se mande un token válido.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - URL: /parking/espacios
- Resultado: El sistema valida el token. Después, consulta la base de datos y devuelve un arreglo en formato JSON con los números de todos los cajones que se encuentran desocupados.

Registrar entrada al estacionamiento:

- Se valida que se pueda registrar la entrada de un vehículo, siempre que se mande un token válido y se cumplan todas las reglas de negocio, incluyendo la comunicación con los otros microservicios.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Cuerpo JSON con: {"idUsuario": 2, "placa": "ABC-1234"}
- Resultado: El sistema valida el token. Después, verifica que haya cajones disponibles y que el usuario no tenga ya 2 vehículos adentro. Luego, se comunica con UserService y VehicleService para confirmar que el usuario y el vehículo son válidos. Si todo es correcto, registra la entrada, ocupa un cajón y devuelve un JSON con los datos del movimiento: idMovimiento, tiempoEntrada, espacioAsignado y tarifaHora.

Registrar salida del estacionamiento:

- Se valida que se pueda registrar la salida de un vehículo que ya se encuentra adentro, liberando su cajón y calculando el costo total del servicio.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Placa enviada en la URL: /parking/salida/ABC-1234
- Resultado: El sistema valida el token y busca el ticket abierto para esa placa. Después, calcula el tiempo transcurrido, las horas cobradas y el costo total basándose en las tarifas de la base de datos. Finalmente, actualiza el ticket, libera el cajón y devuelve un JSON con todos los datos del movimiento, incluyendo tiempoSalida, costoTotal y horasCobradas.



Casos de error

AUTHSERVICE

Usuario inexistente:

- Se valida la restricción de acceso cuando el cliente intenta iniciar sesión ingresando un nombre de usuario que no se encuentra registrado en la base de datos.
- Datos de entrada:
 - Auth Type configurado como No Auth.
 - Cuerpo JSON con: {"username": "123", "password": "admin"}
- Resultado: El sistema ejecuta la consulta a la base de datos basándose en el username proporcionado. Al no localizar ningún registro que coincida, detiene el proceso de inmediato y devuelve un error 400 Bad Request:
 - “El usuario no existe”

Contraseña incorrecta:

- Se valida que el sistema rechace el inicio de sesión si la contraseña ingresada no coincide con la almacenada en la base de datos para ese usuario.
- Datos de entrada:
 - Auth Type configurado como No Auth.
 - Cuerpo JSON con: {"username": "admin", "password": "123"}
- Resultado: El sistema localiza al usuario en la base de datos, pero la herramienta Bcrypt detecta que la contraseña enviada desde Postman no coincide con el hash cifrado. Al detectar la inconsistencia, bloquea la creación del token y devuelve un error 400 Bad Request:
 - “La contraseña es incorrecta”



USERSERVICE

Falta de token (para cualquier función del microservicio):

- Se realiza alguna petición al microservicio, ya sea agregar, editar, ver perfil o cambiar estatus cuando no se proporciona el token en el apartado de Authorization.
- Datos de entrada:
 - Auth Type configurado como No Auth.
- Resultado: El springboot detecta que no se envió el token de autenticación requerido en el header para realizar la operación, entra ni siquiera al código, truena y devuelve una respuesta de error:
 - {"timestamp": "2026-06-16T21:58:14.948+00:00","status": 400,"error": "Bad Request","path": "/usuarios/agregar"}

Token expirado o invalido (para cualquier microservicio):

- Se realiza alguna petición al microservicio, ya sea agregar, editar, ver perfil o cambiar estatus proporcionando un token erróneo o que ya expiró
- Datos de entrada:
 - Auth Type configurado como Auth type Bearer Token.
 - En el apartado de Token (expirado):
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJNb25jaGlzMTIzIiwiaWRVc3VhcmVlIjozLCJpZGFvbm90IiwiaWF0Ij0iMj02Nj00ODgzNDI9.00jwLEItS-p-MGeb0qPEQOvqz8hoo-Qw46KljQ1wFUK
 - En el apartado de Token (erróneo): token_invalidojejejeje
- Resultado: El sistema ingresa el token y pasa por la validación correspondiente, al estar mal el token, el sistema lo detecta y devuelve una respuesta de error:
 - “No pudo completarse: El token es inválido o ya expiró”

El usuario no es administrador (aplica para Agregar usuario y Cambiar estatus):

- Se realiza una petición al microservicio donde se requiere que el usuario autenticado tenga permisos de administrador y se intenta realizar con el token de un usuario que no es admin.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (token de usuario normal):
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbjEiLCJpZGFvbm90IiwiaWF0Ij0iMj02Nj00ODgzNDI9.00jwLEItS-p-MGeb0qPEQOvqz8hoo-Qw46KljQ1wFUK
- Resultado: El sistema valida que el token sea correcto y después busca en la base de datos al usuario autenticado. Al detectar que el usuario no tiene rol de administrador, no permite realizar la operación y devuelve una respuesta de error según la función utilizada:
En Agregar usuario:
“No pudo completarse: ERROR NO ERES ADMIN, NO PUEDES AGREGAR USUARIOS”
En Cambiar estatus:
“No pudo completarse: ERROR NO ERES ADMIN, NO PUEDES CAMBIAR EL ESTATUS DE USUARIOS”

Falta de idUsuario o idUsuario incorrecto (Editar usuario, Ver perfil y Cambiar estatus):



- Se realiza una petición al microservicio donde se necesita enviar el idUsuario en la URL, pero este no se proporciona correctamente o se manda un idUsuario que no existe en la base de datos.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno valido).
 - En la URL no se envía el idUsuario o se envía un idUsuario inexistente.
 - Editar usuario sin idUsuario:
- Resultado: El sistema valida que el idUsuario sea enviado correctamente en la URL. Si el idUsuario no se manda o se manda vacío, Spring Boot puede regresar un error 400 Bad Request o no encontrar la ruta, ya que el parámetro es obligatorio dentro del path. En cambio, si el idUsuario sí se manda pero no existe en la base de datos, el sistema entra a la validación correspondiente y devuelve una respuesta de error según la función utilizada:
 - En Editar usuario:
"No pudo completarse: El usuario que quieres editar no existe"
 - En Ver perfil:
"No pudo completarse: El usuario no existe"
 - En Cambiar estatus:
"No pudo completarse: El usuario no existe"

Falta de idRol o idRol incorrecto (Agregar usuario, Editar usuario y Cambiar estatus):

- Se realiza una petición al microservicio donde se necesita enviar el idRol, pero este no se proporciona correctamente, se manda vacío, se manda un idRol que no existe en el catálogo de roles o no corresponde con el usuario que se quiere actualizar.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - Para Agregar usuario y Editar usuario, el idRol no se envía dentro del cuerpo JSON.
 - Para Cambiar estatus, el idRol no se envía como parámetro en la URL.
- Resultado: El sistema valida que el idRol sea enviado en la URL. En Agregar usuario y Editar usuario, se revisa que venga en el cuerpo JSON y después valida que exista en el catálogo de roles. En Cambiar estatus, el sistema valida que el idRol venga en la URL y que corresponda con el rol real del usuario que se quiere actualizar. Si no se cumple algo de lo anterior, devuelve una respuesta de error según la función utilizada:
 - En Agregar usuario, si falta el idRol:
"No pudo completarse: Falta asignar el idRol del usuario del nuevo usuario"
 - En Editar usuario, si falta el idRol:
"No pudo completarse: Falta asignar el idRol del usuario."
 - En Agregar usuario o Editar usuario, si el idRol no existe:
"No pudo completarse: El id del rol que ingresaste no se encuentra esta asignado a ningún rol"
 - o En Cambiar estatus, si el idRol no corresponde al usuario:
"No pudo completarse: El idRol enviado no corresponde al usuario que quieres actualizar"

Falta de idTipoUsuario o idTipoUsuario incorrecto (Agregar usuario y Editar usuario):



- se hace la petición al microservicio donde se necesita enviar el idTipoUsuario en el cuerpo JSON, pero no se proporciona o se manda un idTipoUsuario que no existe en el catálogo de tipos de usuario.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token(uno válido).
 - Para Agregar usuario y Editar: idTipoUsuario en el JSON.
- Resultado: El sistema valida que el idTipoUsuario sea enviado dentro del cuerpo JSON. Primero revisa que el dato no venga vacío y después valida que exista en el catálogo de tipos de usuario. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - Agregar usuario, si falta el idTipoUsuario:
 - “No pudo completarse: Falta asignar el IdTipoUsuario del nuevo usuario”
 - En Editar usuario, si falta el idTipoUsuario:
 - “No pudo completarse: Falta asignar el IdTipoUsuario del usuario.”
 - En Agregar usuario o Editar usuario, si el idTipoUsuario no existe:
 - “No pudo completarse: El id que proporcionaste no está asignado a ningún tipo de usuario”

Falta de idProgramaEducativo o idProgramaEducativo incorrecto (usuario y Editar usuario):

- Se hace la petición al microservicio donde se necesita enviar el idProgramaEducativo dentro del cuerpo JSON, pero no se proporciona o se manda un idProgramaEducativo que no existe en el catálogo de programas educativos.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - Para Agregar usuario y Editar usuario: idProgramaEducativo dentro del cuerpo JSON.
- Resultado: El sistema valida que el idProgramaEducativo sea enviado dentro del cuerpo JSON. Primero revisa que el dato no venga vacío y después valida que exista en el catálogo de programas educativos. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - En Agregar usuario, si falta el idProgramaEducativo:
 - “No pudo completarse: Falta asignar el programa educativo del nuevo usuario.”
 - En Editar usuario, si falta el idProgramaEducativo:
 - “No pudo completarse: Falta asignar el programa educativo del usuario.”
 - En Agregar usuario o Editar usuario, si el idProgramaEducativo no existe:
 - “No pudo completarse: El id de Programa Educativo ingresado no se encuentra en nuestro catalogo”

Falta de nombre o nombre con demasiados caracteres (Agregar usuario y Editar usuario):

- Se realiza una petición al microservicio donde se necesita enviar el nombre dentro del cuerpo JSON, pero no se proporciona, se manda vacío o supera el tamaño permitido por el sistema
- Datos de entrada:



- Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - Para Agregar usuario y Editar usuario: sin nombre dentro del cuerpo JSON.
 - Para Agregar usuario y Editar usuario: nombre dentro del cuerpo JSON mayor a 50 caracteres.
- Resultado: El sistema valida que el nombre sea enviado dentro del cuerpo JSON. Primero revisa que el dato no venga vacío y después valida que no supere los 50 caracteres permitidos. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - En Agregar usuario, si falta el nombre: “No pudo completarse: Falta asignar el nombre del nuevo usuario.”
 - En Editar usuario, si falta el nombre: “No pudo completarse: Falta asignar el nombre del usuario.”
 - En Agregar usuario o Editar usuario, si el nombre es demasiado largo: “No pudo completarse: El nombre es demasiado largo.”

Falta de apellido paterno o apellido paterno con demasiados caracteres (Agregar usuario y Editar usuario):

- Se hace la petición al microservicio donde se necesita enviar el apellido paterno dentro del cuerpo JSON, pero no se proporciona, se manda vacío o supera el tamaño permitido por el sistema.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - Para Agregar usuario y Editar usuario: sin apellidoPaterno dentro del cuerpo JSON mayor a 50 caracteres.
- Resultado: El sistema valida que el apellido paterno sea enviado dentro del cuerpo JSON. Primero revisa que el dato no venga vacío y después valida que no supere los 50 caracteres permitidos. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - En Agregar usuario, si falta el apellido paterno: “No pudo completarse: Falta asignar el apellido paterno del nuevo usuario.”
 - En Editar usuario, si falta el apellido paterno: “No pudo completarse: Falta asignar el apellido paterno del usuario.”
 - En Agregar usuario o Editar usuario, si el apellido paterno es demasiado largo: “No pudo completarse: El apellido paterno es demasiado largo.”

Falta de email, email incorrecto, email repetido o email con demasiados caracteres (Agregar usuario y Editar usuario):

- Se hace una petición al microservicio donde se necesita enviar el email dentro del cuerpo JSON, pero no se proporciona, se manda vacío, tiene un formato incorrecto, ya se encuentra registrado o supera el tamaño permitido por el sistema.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).

- Para Agregar usuario y Editar usuario: sin email dentro del cuerpo JSON.
- Para Agregar usuario y Editar usuario: "email": "johaneditado-SINARROBA_j2rp.com",
- Para Agregar usuario y Editar usuario: "email": "correoaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaa@j2rp.com"
- Resultado: El sistema valida que el email sea enviado dentro del cuerpo JSON. Primero revisa que el dato no venga vacío, después valida que no supere los 100 caracteres, que tenga un formato válido y que no exista otro usuario registrado con el mismo correo. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - En Agregar usuario, si falta el email:
“No pudo completarse: Falta asignar el email del nuevo usuario”
 - En Editar usuario, si falta el email:
“No pudo completarse: Falta asignar el email del usuario”
 - En Agregar usuario o Editar usuario, si el email tiene un formato incorrecto:
“No pudo completarse: El correo proporcionado no tiene el formato valido”
 - En Agregar usuario o Editar usuario, si el email es demasiado largo:
“No pudo completarse: El correo es demasiado largo.”
 - En Agregar usuario o Editar usuario, si el email ya está registrado:
“No pudo completarse: El correo electrónico ya se encuentra registrado”

Falta de teléfono o teléfono con demasiados caracteres (Agregar usuario y Editar usuario):

- Se realiza una petición al microservicio donde se necesita enviar el teléfono dentro del cuerpo JSON, pero no se proporciona, se manda vacío o supera el tamaño permitido por el sistema.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - Para Agregar usuario y Editar usuario: sin teléfono dentro del cuerpo JSON.
 - Para Agregar usuario y Editar usuario: "telefono": "2281234567891011"
- Resultado: El sistema valida que el teléfono esté en el cuerpo JSON. Primero revisa que el dato no venga vacío y después valida que no supere los 13 caracteres permitidos. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según la función utilizada:
 - o En Agregar usuario, si falta el teléfono: "No pudo completarse: Falta asignar el teléfono del nuevo usuario."
 - o En Editar usuario, si falta el teléfono: "No pudo completarse: Falta asignar el teléfono del usuario."
 - o En Agregar usuario o Editar usuario, si el teléfono es demasiado largo: "No pudo completarse: El teléfono es demasiado largo."

Falta de username o username repetido (Agregar usuario):



- Se hace la petición al microservicio donde se necesita enviar el username dentro del cuerpo JSON, pero no se proporciona, se manda vacío o ya existe otro usuario registrado con el mismo username.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token se coloca un token válido de administrador.
 - Para Agregar usuario: sin username dentro del cuerpo JSON.
 - Para Agregar usuario: "username": "johandrade",
- Resultado: El sistema valida que el username venga dentro del JSON. Primero revisa que el dato no venga vacío y después busca en la base de datos si ya hay algún usuario con el mismo user. Si no se cumple alguna de estas condiciones, devuelve una respuesta de error según el caso:
 - En Agregar usuario, si falta el username: "No pudo completarse: Falta asignar el username del nuevo usuario."
 - En Agregar usuario, si el username ya está registrado: "No pudo completarse: El nombre de usuario ya está en uso"

Falta de password (Agregar usuario):

- Se realiza una petición al microservicio donde se necesita enviar la password dentro del cuerpo JSON, pero no se proporciona o se manda vacía.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token se coloca un token válido de administrador.
 - Para Agregar usuario: sin password dentro del cuerpo JSON.
- Resultado: El sistema valida que el password sea enviado dentro del cuerpo JSON. Si el dato no viene o se manda vacío, no permite registrar al usuario y devuelve una respuesta de error:
 - En Agregar usuario, si falta la password: "No pudo completarse: Falta asignar el password del nuevo usuario."

Falta de idUsuario o idUsuario incorrecto como path param (Editar usuario, Ver perfil y Cambiar estatus):

- Se lanza la petición al microservicio donde se necesita enviar el idUsuario en la URL, pero no se proporciona, se manda vacío o se manda un idUsuario que no existe en la base de datos.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token (uno válido).
 - En la URL no se envía el idUsuario o se envía un idUsuario inexistente.
- Resultado: El sistema valida que el idUsuario sea enviado correctamente dentro de la URL. Si el idUsuario no se manda, se manda vacío o se manda con un formato incorrecto, Spring Boot devuelve un error 400 Bad Request y no encuentra la ruta. En cambio, si el idUsuario



sí se manda pero no existe en la base de datos, el sistema entra a la validación correspondiente y devuelve una respuesta de error según la función utilizada:

- Para todos los servicios:
 - {
 - "timestamp": "2026-06-17T01:40:20.688+00:00",
 - "status": 404,
 - "error": "Not Found",
 - "path": usuarios/agregar
- }
- En Editar usuario:
"No pudo completarse: El usuario que quieres editar no existe"
- En Ver perfil:
"No pudo completarse: El usuario no existe"
- En Cambiar estatus:
"No pudo completarse: El usuario no existe"

Falta de idRol o idRol incorrecto como path param (Cambiar estatus):

- Se hace la petición al microservicio donde se necesita enviar el idRol en la URL para cambiar el estatus de un usuario, pero no se proporciona, se manda vacío o se manda un idRol que no corresponde con el usuario que se quiere actualizar.
- Datos de entrada:
 - Auth Type configurado como Bearer Token.
 - En el apartado de Token se coloca un token válido de administrador.
 - En la URL no se envía el idRol o se envía un idRol que no es del usuario.
- Resultado: El sistema valida que el idRol este en la URL. Si el idRol no se manda o se manda vacío, Spring Boot devuelve 400 Bad Request por no encontrar la ruta. Si el idRol sí se manda pero no corresponde con el rol real del usuario que se quiere actualizar, el sistema entra a la validación correspondiente y devuelve:
 - "No pudo completarse: El idRol enviado no corresponde al usuario que quieres actualizar"



VEHICLESERVICE

Registrar placa repetida:

- Se valida que el sistema no permita registrar un vehículo con una placa que ya existe en la base de datos.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Cuerpo json con: {"idUsuario": 2, "idModelo": 1, "placa": "ABC1234", "color": "Rojo", "anio": 2020, "descripcion": "Vehiculo de prueba repetido"}
- Resultado: El sistema valida el token y los datos del body. Después busca si ya existe un vehículo con la misma placa. Como la placa ya se encuentra registrada, detiene la operación y devuelve:
 - "La placa ya está registrada."

Petición sin token:

- Se valida que el sistema no permita consumir los endpoints protegidos si no se manda un token de autorización.
- Datos de entrada:
 - No se manda token en Auth Type.
 - idUsuario enviado en la URL: /vehiculos/buscarPorUsuario/2
- Resultado: El sistema detecta que no se mandó el token de autorización y rechaza la petición. La respuesta nos muestra que falta el token de autorización y devuelve un error de:
 - Bad Request.

Registrar más de 4 vehículos activos:

- Se valida que el sistema no permita registrar más de cuatro vehículos activos asociados al mismo usuario.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Usuario con cuatro vehículos activos registrados.
 - Cuerpo JSON con: {"idUsuario": 2, "idModelo": 5, "placa": "DDD4444", "color": "Azul", "anio": 2024, "descripcion": "Quinto vehiculo activo"}
- Resultado: El sistema valida el token, los datos obligatorios, el modelo y la placa. Después cuenta cuántos vehículos activos tiene el usuario. Como el usuario ya tiene cuatro vehículos activos, detiene el registro y devuelve:
 - "El usuario ya tiene cuatro vehículos activos."

Validar vehículo inactivo para ParkingService:

- Se valida que el sistema no permita validar un vehículo para ParkingService cuando el vehículo se encuentra inactivo.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Vehículo cambiado a estatus inactivo.



- idUsuario y placa enviados como parámetros en la URL:
/vehiculos/validar?idUsuario=2&placa=XYZ9876
- Resultado: El sistema valida el token, el idUsuario y la placa. Después busca si existe un vehículo activo asociado a ese usuario y placa. Como el vehículo se encuentra inactivo, no permite la validación y devuelve:
 - “El vehículo no existe, no está asignado al usuario o esta inactivo.”



PARKINGSERVICE

Registrar entrada de vehículo ya estacionado:

- Se valida que el sistema no permita registrar la entrada de un vehículo que ya cuenta con un acceso activo (que ya está adentro del estacionamiento).
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Cuerpo JSON con un vehículo que ya ingresó previamente: {"idUsuario": 2, "placa": "ABC1234"}
- Resultado: El sistema valida el token, la disponibilidad de cajones y el límite por usuario. Después busca si la placa ya tiene un ticket abierto. Como el vehículo ya se encuentra adentro, detiene la operación y devuelve un error de Bad Request con el mensaje:
 - "El vehículo con placa ABC1234 ya se encuentra adentro del estacionamiento."

Petición con token inválido o expirado:

- Se valida que el sistema no permita consumir los endpoints si el token de autorización enviado fue modificado, es falso o ya expiró su tiempo de vida.
- Datos de entrada:
 - Token con caracteres modificados a propósito, colocado en Auth Type Bearer.
 - Petición GET enviada a la URL: /parking/espacios
- Resultado: El sistema extrae el token y el service intenta verificar matemáticamente la firma usando la llave secreta compartida. Al detectar que la firma no coincide o ya caducó, rechaza la petición y devuelve un error de Bad Request con el mensaje:
 - "No pudo completarse: El token es inválido o ya expiró."

Registrar más de 2 vehículos adentro por usuario:

- Se valida que el sistema no permita que un mismo usuario ingrese más de dos vehículos al estacionamiento de manera simultánea.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Usuario que ya tiene dos vehículos adentro del estacionamiento.
 - Cuerpo JSON con el intento de ingreso de un tercer vehículo: {"idUsuario": 2, "placa": "XYZ9876"}
- Resultado: El sistema valida el token y verifica que hay cajones disponibles. Después cuenta cuántos vehículos activos tiene ese usuario dentro del estacionamiento. Como el usuario ya alcanzó su límite de 2, detiene el registro y devuelve un error de Bad Request con el mensaje:
 - "El usuario ya tiene el máximo permitido de 2 vehículos adentro."

Registrar entrada con vehículo ajeno o inactivo (Error de Integración):

- Se valida que el sistema bloquee el acceso si la placa ingresada no existe, está inactiva o pertenece a un usuario diferente, apoyándose en la comunicación con VehicleService.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.



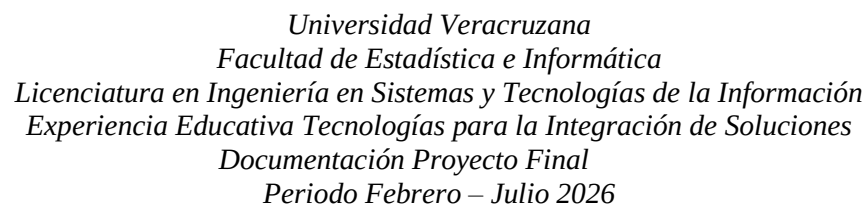
- Cuerpo JSON con una placa inventada o que pertenece a otro usuario: {"idUsuario": 2, "placa": "ZZZ0000"}
- Resultado: El sistema valida las reglas locales (cajones y límites) y luego hace una petición mediante RestTemplate a VehicleService enviando el token. Al recibir un rechazo desde el otro microservicio, detiene la entrada y devuelve un error de Bad Request con el mensaje:
 - "No pudo completarse: Error de Integración: El vehículo no existe, no le pertenece al usuario o está inactivo."

Registrar salida de vehículo no estacionado:

- Se valida que el sistema no permita registrar la salida ni generar un cobro a una placa que no tiene un registro de entrada activo en ese momento.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Placa de un vehículo que no está adentro enviada en la URL: /parking/salida/XYZ9876
- Resultado: El sistema valida el token y busca en la base de datos si existe un ticket abierto (estatus 1) asociado a esa placa. Al no encontrar ningún registro activo, detiene la operación y devuelve un error de Bad Request con el mensaje:
 - "No pudo completarse: No se encontró ningún vehículo adentro con la placa XYZ9876"

Registrar entrada con estacionamiento lleno:

- Se valida que el sistema bloquee nuevas entradas y no asigne lugares cuando todos los cajones de la base de datos se encuentran ocupados.
- Datos de entrada:
 - Token obtenido de AuthService y colocado en Auth Type Bearer.
 - Cuerpo JSON con un vehículo válido: {"idUsuario": 3, "placa": "QWE1111"}
- Resultado: El sistema valida el token. Después busca en la tabla de cajones si existe algún espacio con el estatus desocupado. Al confirmar que no hay ningún idEspacio libre, detiene la operación inmediatamente y devuelve un error de Bad Request con el mensaje:
 - "No pudo completarse: Estacionamiento lleno. No hay cajones disponibles."



Evidencia 1. Login exitoso



Ilustración 6 Evidencia de authService numero 2

52

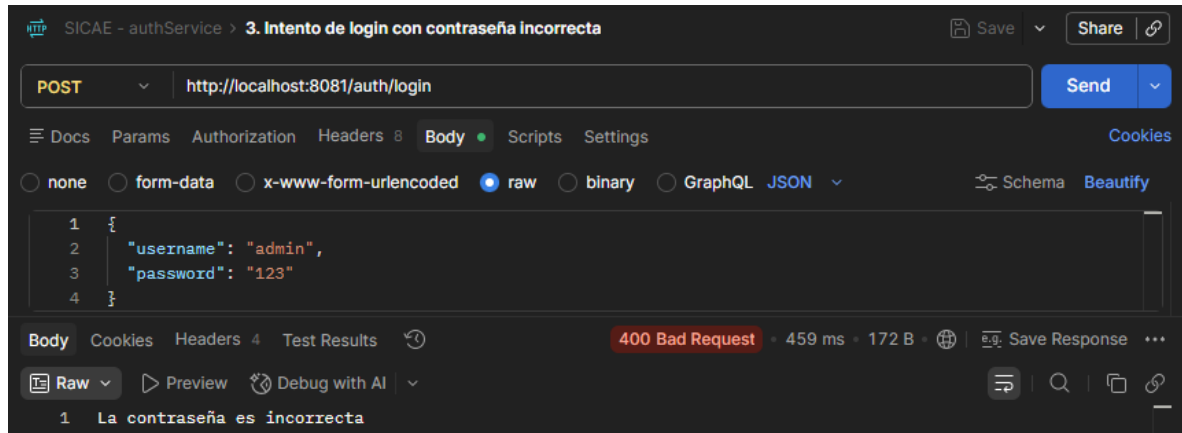


Ilustración 7 Evidencia de authService numero 3



USERSERVICE

Agregar usuario:

http://localhost:8082/usuarios/agregar

POST

Body

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 3,
4   "idProgramaEducativo": 1,
5   "nombre": "Prueba",
6   "apellidoPaterno": "Prueba",
7   "apellidoMaterno": "Registro",
8   "email": "juan.registro001@esicae.com",
9   "telefono": "2281234567",
10  "username": "juanregistro001",
11  "password": "Password123"
12 }
```

200 OK • 202 ms • 232 B

Operación realizada correctamente: El usuario se agregó con éxito

Ilustración 8 Evidencia de userService numero 1

Editar usuario:

UserService > EditarUsuario

PUT

Body

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan Editado",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan.editado@j2xp.com",
9   "telefono": "2281234567"
10 }
```

200 OK • 28 ms • 231 B

Operación realizada correctamente: El usuario se editó con éxito

Ilustración 9 Evidencia de userService numero 2

Ver perfil de usuario:



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService > VerPerfil

GET http://localhost:8082/usuarios/verPerfil/3

Docs Params Authorization Headers 7 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 5 Test Results

200 OK • 10 ms • 490 B • Save Response

```
{
  "idUsuario": 3,
  "rol": 2,
  "nombre": "Monseriat",
  "apellidoPaterno": "Hernandez",
  "apellidoMaterno": "Nieto",
  "tipoUsuario": 3,
  "programaEducativo": 1,
  "usuario": "Monchis123",
  "correo": "MonSHN@sicae.com",
  "telefono": "2281234567",
  "estatus": true,
  "claveUsuario": "PRU-124",
  "tiempoCreacion": "2026-06-12T05:58:23.682914",
  "tiempoActualizacion": null
}
```

Ilustración 10 Evidencia de userService numero 3

Cambiar estatus de usuario:

UserService > CambiarEstatus

PATCH http://localhost:8082/usuarios/actualizarEstatus/4/2

Docs Params Authorization Headers 8 Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results

200 OK • 26 ms • 247 B • Save Response

Raw Preview Visualize

```
1 Operación realizada correctamente: El estatus del usuario se actualizó con éxito
```

Ilustración 11 Evidencia de userService numero 4



Falta de idUsuario o idUsuario incorrecto (aplica para Editar usuario, Ver perfil y Cambiar estatus):

UserService Copy > EditarUsuario-idUsuarioMal

PUT http://localhost:8082/usuarios/editar/

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan Editado",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan.editado@j2xp.com",
9   "telefono": "2281234567"
10 }
```

Body Cookies Headers 8 Test Results 404 Not Found

JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-06-16T23:21:11.799+00:00",
3   "status": 404,
4   "error": "Not Found",
5   "path": "/usuarios/editar/"
6 }
```

Ilustración 12 Evidencia de userService numero 5

UserService Copy > EditarUsuario-idUsuarioMal

PUT http://localhost:8082/usuarios/editar/8999

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan Editado",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan.editado@j2xp.com",
9   "telefono": "2281234567"
10 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

```
1 No pudo completarse: El usuario que quieres editar no existe
```

Ilustración 13 Evidencia de userService numero 6



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

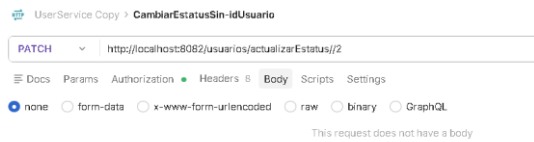


Ilustración 14 Evidencia de userService numero 7

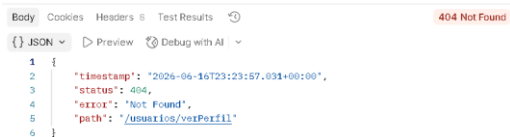
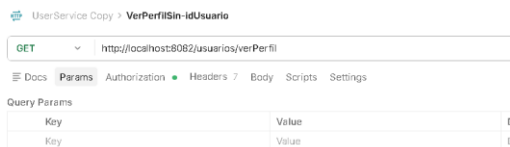


Ilustración 15 Evidencia de userService numero 8

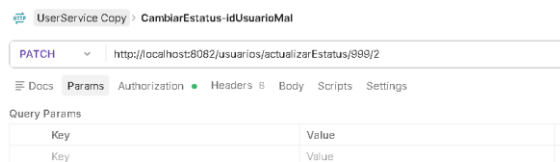


Ilustración 16 Evidencia de userService numero 9



Falta idRol:

UserService Copy > UsuarioSin-id

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idTipoUsuario": 1,
3   "idProgramaEducativo": 1,
4   "nombre": "Johan",
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Merino",
7   "email": "johan@2xp.com",
8   "telefono": "2281234567",
9   "username": "johandrade",
10  "password": "123!"
11 }
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el idRol del usuario del nuevo usuario

Ilustración 17 Evidencia de userService numero 10

UserService Copy > UsuarioEditarSin-idRol

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idTipoUsuario": 1,
3   "idProgramaEducativo": 1,
4   "nombre": "Johan Editado",
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Merino",
7   "email": "johan.editado@2xp.com",
8   "telefono": "2281234567"
9 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el idRol del usuario.

Ilustración 18 Evidencia de userService numero 11

UserService Copy > Usuario-idRolMal

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 22,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan@2xp1.com",
9   "telefono": "2281234567",
10  "username": "johandrade1",
11  "password": "123!"
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request - 20 ms - 23

Raw Preview Debug with AI

1 No pudo completarse: El id del rol que ingresaste no se encuentra esta asignado a ningun rol

Ilustración 19 Evidencia de userService numero 12



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

Falta

idTipoUsuario:

UserService Copy > Usuario-Idtipousuariomal

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   'idRol': 2,
3
4   'idProgramaEducativo': 1,
5   'nombre': 'Johan',
6   'apellidoPaterno': 'Andrade',
7   'apellidoMaterno': 'Merino',
8   'email': 'johan@j2rp1.com',
9   'telefono': '2281234567',
10  'username': 'johandrade1',
11  'password': '1231'
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el IdTipoUsuario del nuevo usuario

Ilustración 20 Evidencia de userService numero 13

UserService Copy > UsuarioEditarSin-idtipousuario

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   'idRol': 2,
3   'idProgramaEducativo': 1,
4   'nombre': 'Johan Editado',
5   'apellidoPaterno': 'Andrade',
6   'apellidoMaterno': 'Merino',
7   'email': 'johan.editado@j2rp.com',
8   'telefono': '2281234567'
9 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el IdTipoUsuario del usuario.

Ilustración 21 Evidencia de userService numero 14

UserService Copy > Usuario-Idtipousuariomal

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   'idRol': 2,
3   'idTipoUsuario': 11,
4   'idProgramaEducativo': 1,
5   'nombre': 'Johan',
6   'apellidoPaterno': 'Andrade',
7   'apellidoMaterno': 'Merino',
8   'email': 'johan@j2rp1.com',
9   'telefono': '2281234567',
10  'username': 'johandrade1',
11  'password': '1231'
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request - 22 m

Raw Preview Debug with AI

1 No pudo completarse: El id que proporcionaste no está asignado a ningún tipo de usuario

Ilustración 22 Evidencia de userService numero 15



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService Copy > UsuarioEditar-ldtipousuariomal

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 11,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan Editado",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan.editado@j2ip.com",
9   "telefono": "2281234567"
10 }
```

Body Cookies Headers 4 Test Results 400 Bad Request 25 ms

Raw Preview Debug with AI

1 No pudo completarse: El id que proporcionaste no está asignado a ningún tipo de usuario

Ilustración 23 Evidencia de userService numero 16

Falta de idProgramaEducativo o idProgramaEducativo incorrecto:

UserService Copy > Usuariosin-idprogramaE

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "nombre": "Johan",
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Merino",
7   "email": "johan@j2ip.com",
8   "telefono": "2281234567",
9   "username": "johandrade",
10  "password": "1231"
11 }
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el programa educativo del nuevo usuario.

Ilustración 24 Evidencia de userService numero 17

UserService Copy > Usuario-idprogramaEmail

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 11,
5   "nombre": "Johan",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan@j2ip.com",
9   "telefono": "2281234567",
10  "username": "johandrade1",
11  "password": "1231"
12 }
```

Body Cookies Headers 4 Test Results 400 Bad Request 23 ms 238

Raw Preview Debug with AI

1 No pudo completarse: El id de Programa Educativo ingresado no se encuentra en nuestro catalogo

Ilustración 25 Evidencia de userService numero 18



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService Copy > UsuarioEditarSin-idprogramaE

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "nombre": "Johan Editado",
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Merino",
7   "email": "johan.editado@j2ip.com",
8   "telefono": "2281234567"
9 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el programa educativo del usuario.

Ilustración 26 Evidencia de userService numero 19

UserService Copy > UsuarioEditar-idprogramaEmail

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers 4 Test Results 400 Bad Request 18 ms 238 B

Raw Preview Debug with AI

1 No pudo completarse: El id de Programa Educativo ingresado no se encuentra en nuestro catalogo

Ilustración 27 Evidencia de userService numero 20

Falta nombre:

UserService > UsuarioSin-nombre

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Merino",
7   "email": "johan@j2ip.com",
8   "telefono": "2281234567",
9   "username": "johandado",
10  "password": "1231"
11 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el nombre del nuevo usuario.

Ilustración 28 Evidencia de userService numero 21



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService > UsuarioEditarSin-nombre

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "apellidoPaterno": "Andrade",
6   "apellidoMaterno": "Marino",
7   "email": "johan.editado@j2ip.com",
8   "telefono": "2281234567"
9 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el nombre del usuario.

Ilustración 29 Evidencia de userService numero 22

UserService > UsuarioEditar-nombreLargo

PUT http://localhost:8082/usuarios/editar/6

Docs Params Authorization Headers 10 Body Scripts Settings

Query Params

Key	Value	D
Key	Value	D

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: El nombre es demasiado largo.

Ilustración 30 Evidencia de userService numero 23

Falta o es muy largo el apellido paterno:

UserService > UsuarioSin-apellidoP

POST http://localhost:8082/usuarios/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

Query Params

Key	Value	C
Key	Value	C

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el apellido paterno del nuevo usuario.

Ilustración 31 Evidencia de userService numero 24



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService > UsuarioEditarSin-apellidoP

PUT http://localhost:8082/usuarios/editar/6

Params Authorization Headers 10 Body Scripts Settings

Query Params

Key	Value	Display
Key	Value	Display

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el apellido paterno del usuario.

Ilustración 32 Evidencia de userService numero 25

UserService > UsuarioEditar-apellidoPLargo

PUT http://localhost:8082/usuarios/editar/6

Params Authorization Headers 10 Body Scripts Settings

Query Params

Key	Value	Display
Key	Value	Display

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: El apellido paterno es demasiado largo.

Ilustración 33 Evidencia de userService numero 26

Usuario sin correo, mal, usado, largo:

UserService > UsuarioSin-correo

POST http://localhost:8082/usuarios/agregar

Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

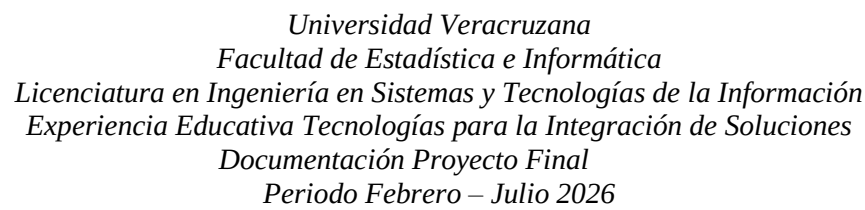
```
1 {
2   'idRol': 2,
3   'idTipoUsuario': 1,
4   'idProgramaEducativo': 1,
5   'nombre': 'Johan',
6   'apellidoPaterno': 'Andrade',
7   'apellidoMaterno': 'Maximo',
8   'telefono': '2281234567',
9   'username': 'johandrade',
10  'password': '123!'
11 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: Falta asignar el email del nuevo usuario

Ilustración 34 Evidencia de userService numero 27





Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

Usuario sin telefono o demasiado largo:

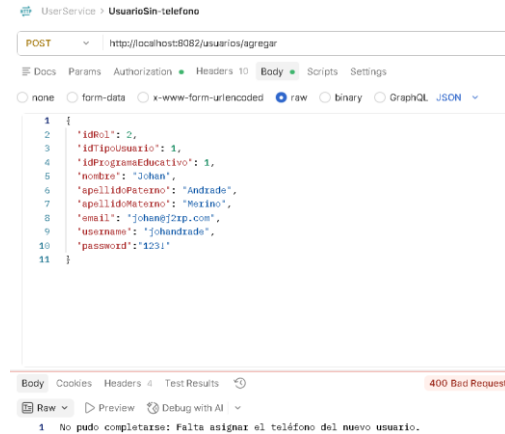


Ilustración 38 Evidencia de userService numero 31

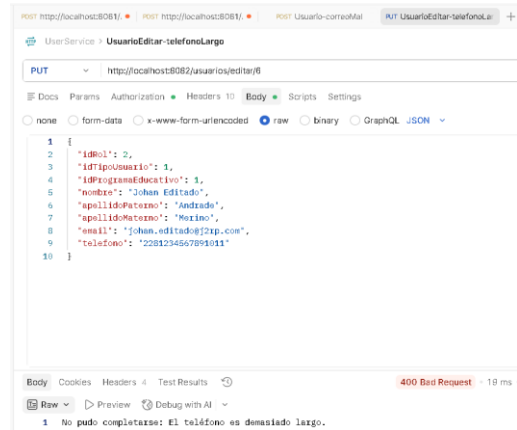


Ilustración 39 Evidencia de userService numero 32

Usuario sin username o username repetido

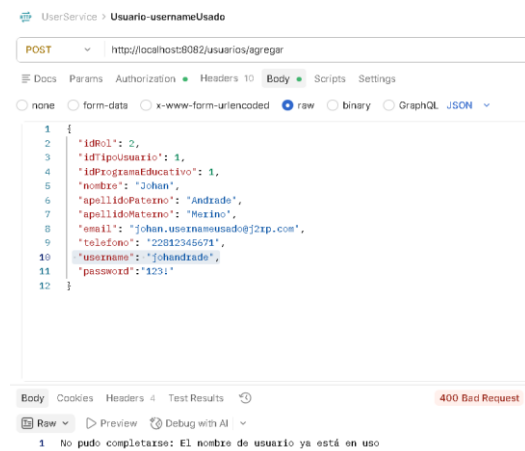


Ilustración 40 Evidencia de userService numero 33



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

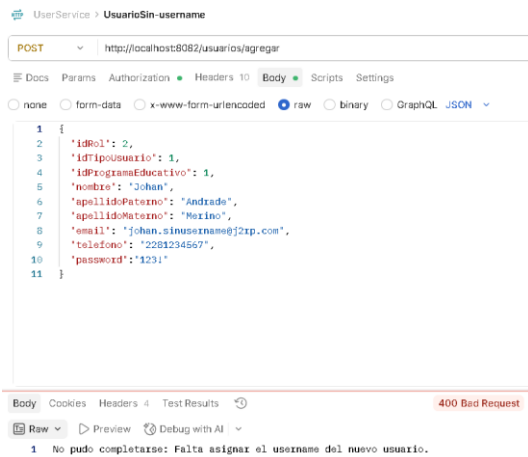


Ilustración 41 Evidencia de userService numero 34

Usuario sin password

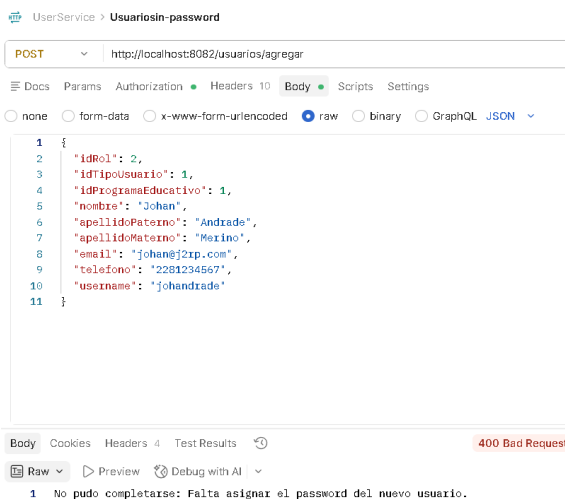


Ilustración 42 Evidencia de userService numero 35

Falta idUsuario como pathparam

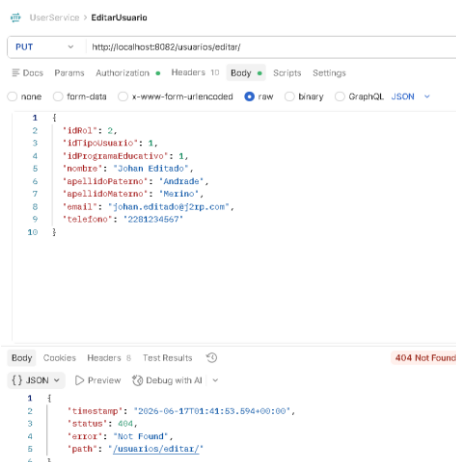


Ilustración 43 Evidencia de userService numero 36



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

UserService > EditarUsuario

PUT http://localhost:8082/usuarios/editar/999

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "idRol": 2,
3   "idTipoUsuario": 1,
4   "idProgramaEducativo": 1,
5   "nombre": "Johan Editado",
6   "apellidoPaterno": "Andrade",
7   "apellidoMaterno": "Merino",
8   "email": "johan.editado@j2xp.com",
9   "telefono": "2281234567"
10 }
```

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: El usuario que quieres editar no existe

Ilustración 44 Evidencia de userService numero 37

UserService > VerPerfil

GET http://localhost:8082/usuarios/verPerfil/99

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	D
Key	Value	D

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: El usuario no existe

Ilustración 45 Evidencia de userService numero 38

UserService > CambiarEstatus

PATCH http://localhost:8082/usuarios/actualizarEstatus/66/2

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

Body Cookies Headers 4 Test Results 400 Bad Request

Raw Preview Debug with AI

1 No pudo completarse: El usuario no existe

Ilustración 46 Evidencia de userService numero 39



Universidad Veracruzana
Facultad de Estadística e Informática
Licenciatura en Ingeniería en Sistemas y Tecnologías de la Información
Experiencia Educativa Tecnologías para la Integración de Soluciones
Documentación Proyecto Final
Periodo Febrero – Julio 2026

Cambiar **estatus** **sin** **idRol** **o** **idRol** **erróneo:**

UserService > CambiarEstatusSin-idRol

PATCH http://localhost:8082/usuarios/actualizarEstatus/4/

Docs Params Authorization Headers Body Scripts Settings

Query Params

Key	Value	De
Key	Value	De

Body Cookies Headers Test Results 404 Not Found

JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-06-17T01:49:42.351+00:00",
3   "status": 404,
4   "error": "Not Found",
5   "path": "/usuarios/actualizarEstatus/4/"
6 }
```

Ilustración 47 Evidencia de userService numero 40

UserService > CambiarEstatus-idRolMal

PATCH http://localhost:8082/usuarios/actualizarEstatus/4/1

Docs Params Authorization Headers Body Scripts Settings

Query Params

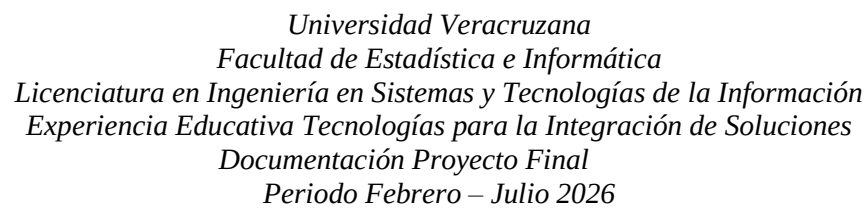
Key	Value	Descript
Key	Value	Descript

Body Cookies Headers Test Results 400 Bad Request 15 m

Raw Preview Debug with AI

```
1 No pudo completarse: El idRol enviado no corresponde al usuario que quieres actualizar
```

Ilustración 48 Evidencia de userService numero 41



Evidencia 1. Login exitoso en AuthService.



Ilustración 50 Evidencia de vehicleService numero 2



Evidencia 3. Consulta de vehículos por usuario.

SICAE - VehicleService > 03 - Buscar vehículos por usuario

GET http://localhost:8083/vehiculos/buscarPorUsuario/2

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does not have a body

200 OK · 89 ms · 374 B

Save Response

Body Cookies Headers 5 Test Results

JSON Preview Visualize

```
1 {
2   {
3     "idVehiculo": 1,
4     "idUsuario": 2,
5     "claveVehiculo": "V-1",
6     "idMarca": 1,
7     "marca": "Toyota",
8     "idModelo": 1,
9     "modelo": "Corolla",
10    "placa": "ABC1234",
11    "color": "Rojo",
12    "anio": 2020,
13    "estatus": true,
14    "descripcion": "vehículo pa probar"
15  }
16 }
```

Ilustración 51 Evidencia de vehicleService numero 3

Evidencia 4. Rechazo por placa repetida.

SICAE - VehicleService > 04 - Registrar placa repetida

POST http://localhost:8083/vehiculos/agregar

Docs Params Authorization Headers 10 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   {
3     "idUsuario": 2,
4     "idModelo": 1,
5     "placa": "ABC1234",
6     "color": "Rojo",
7     "anio": 2020,
8     "descripcion": "Vehículo pa probar"
9   }
10 }
```

400 Bad Request · 129 ms · 193 B

Save Response

Body Cookies Headers 4 Test Results

Raw Preview Debug with AI

```
1 No se pudo realizar: La placa ya esta registrada.
```

Ilustración 52 Evidencia de vehicleService numero 4



Evidencia 5. Edición exitosa de vehículo.

The screenshot shows a REST client interface for a service named 'SICAE - VehicleService'. The selected endpoint is '05 - Editar vehículo' with a PUT method. The URL is 'http://localhost:8083/vehiculos/editar/1'. The request body is a JSON object:

```
{  "idUsuario": 2,  "idModelo": 4,  "placa": "XYZ9876",  "color": "Azul",  "anio": 2021,  "descripcion": "vehículo editado"}
```

. The response status is '200 OK' with a response time of 113 ms and a body size of 202 B. The response body contains the message: '1 El vehículo se actualizo correctamente'.

Ilustración 53 Evidencia de vehicleService numero 5

Evidencia 6. Cambio de estatus.

The screenshot shows a REST client interface for the same 'SICAE - VehicleService'. The selected endpoint is '06 - Cambiar estatus' with a PATCH method. The URL is 'http://localhost:8083/vehiculos/actualizarEstatus/1/2'. The request body is empty, indicated by the message 'This request does not have a body'. The response status is '200 OK' with a response time of 28 ms and a body size of 205 B. The response body contains the message: '1 estatus del vehículo cambiado con éxito!!'.

Ilustración 54 Evidencia de vehicleService numero 6



Evidencia 7. Validación exitosa para ParkingService.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8083/vehiculos/validar?idUsuario=2&placa=XYZ9876`
- Method:** GET
- Body:** none (selected)
- Response:** 200 OK, 35 ms, 367 B. The response body is a JSON object:

```
{  "idVehiculo": 1,  "idUsuario": 2,  "claveVehiculo": "V-1",  "idMarca": 2,  "marca": "Honda",  "idModelo": 4,  "modelo": "Civic",  "placa": "XYZ9876",  "color": "Azul",  "anio": 2021,  "estatus": true,  "descripcion": "vehiculo editado"}
```

Ilustración 55 Evidencia de vehicleService numero 7

Evidencia 8. Rechazo de petición sin token.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8083/vehiculos/buscarPorUsuario/2`
- Method:** GET
- Auth Type:** No Auth (selected)
- Response:** 400 Bad Request, 14 ms, 263 B. The response body is a JSON object:

```
{  "timestamp": "2026-06-14T05:16:55.604+00:00",  "status": 400,  "error": "Bad Request",  "path": "/vehiculos/buscarPorUsuario/2"}
```

Ilustración 56 Evidencia de vehicleService numero 8



Evidencia 9. Rechazo por superar el máximo de 4 vehículos activos.

HTTP SICAE - VehicleService > 09 - Registrar quinto vehículo activo

POST http://localhost:8083/vehiculos/agregar

Body

```
1 {
2   "idUserario": 2,
3   "idModelo": 2,
4   "placa": "QWE1234",
5   "color": "Blanco",
6   "anio": 2022,
7   "descripcion": "Quinto vehículo de prueba"
8 }
```

400 Bad Request • 22 ms • 211 B

1 No se pudo realizar: El usuario ya tiene cuatro vehiculos activos.

Ilustración 57 Evidencia de vehicleService numero 9



PARKINGSERVICE

Entrada de auto a estacionamiento:

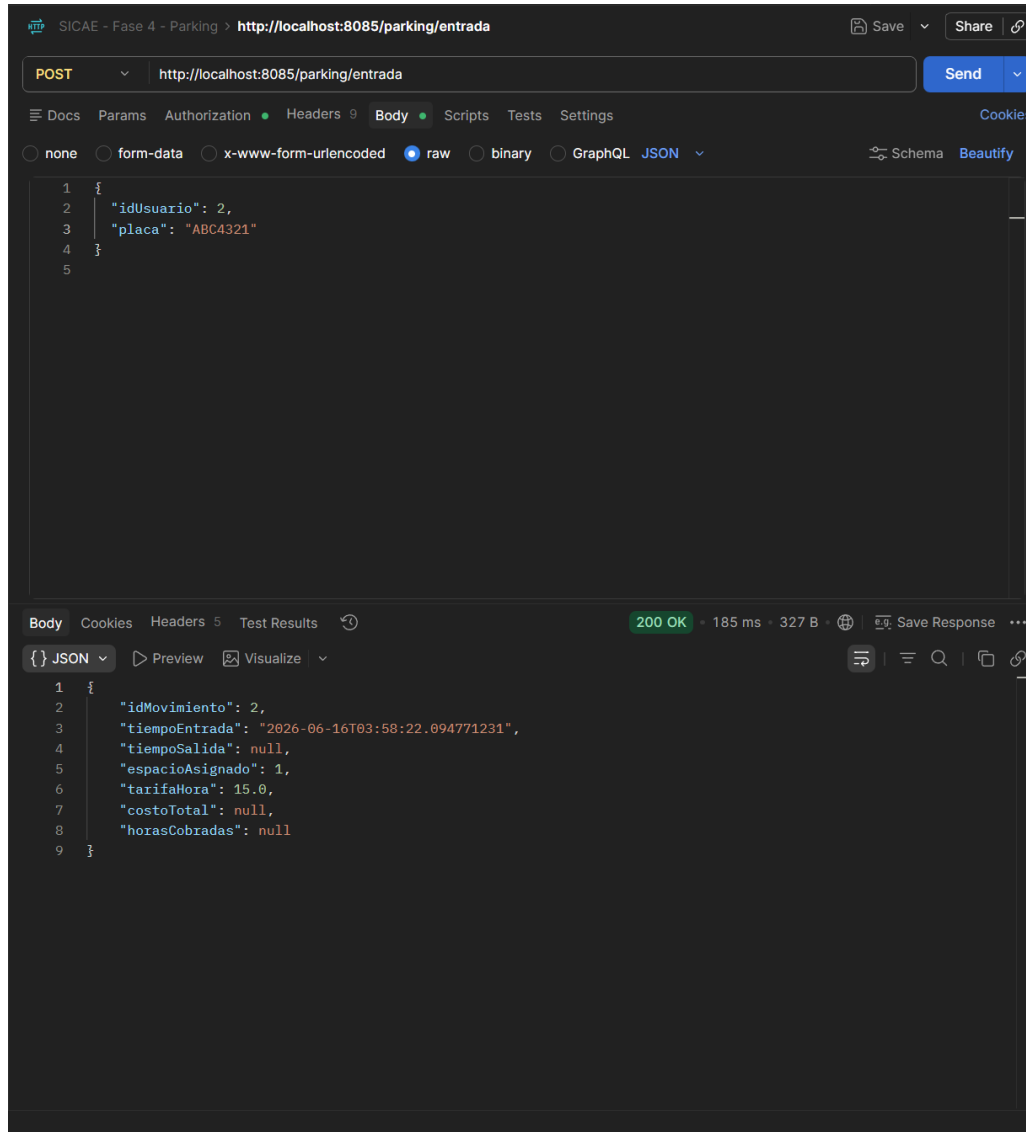


Ilustración 58 Evidencia de parkingService numero 1



Ver los cajones disponibles después de agregar el auto:

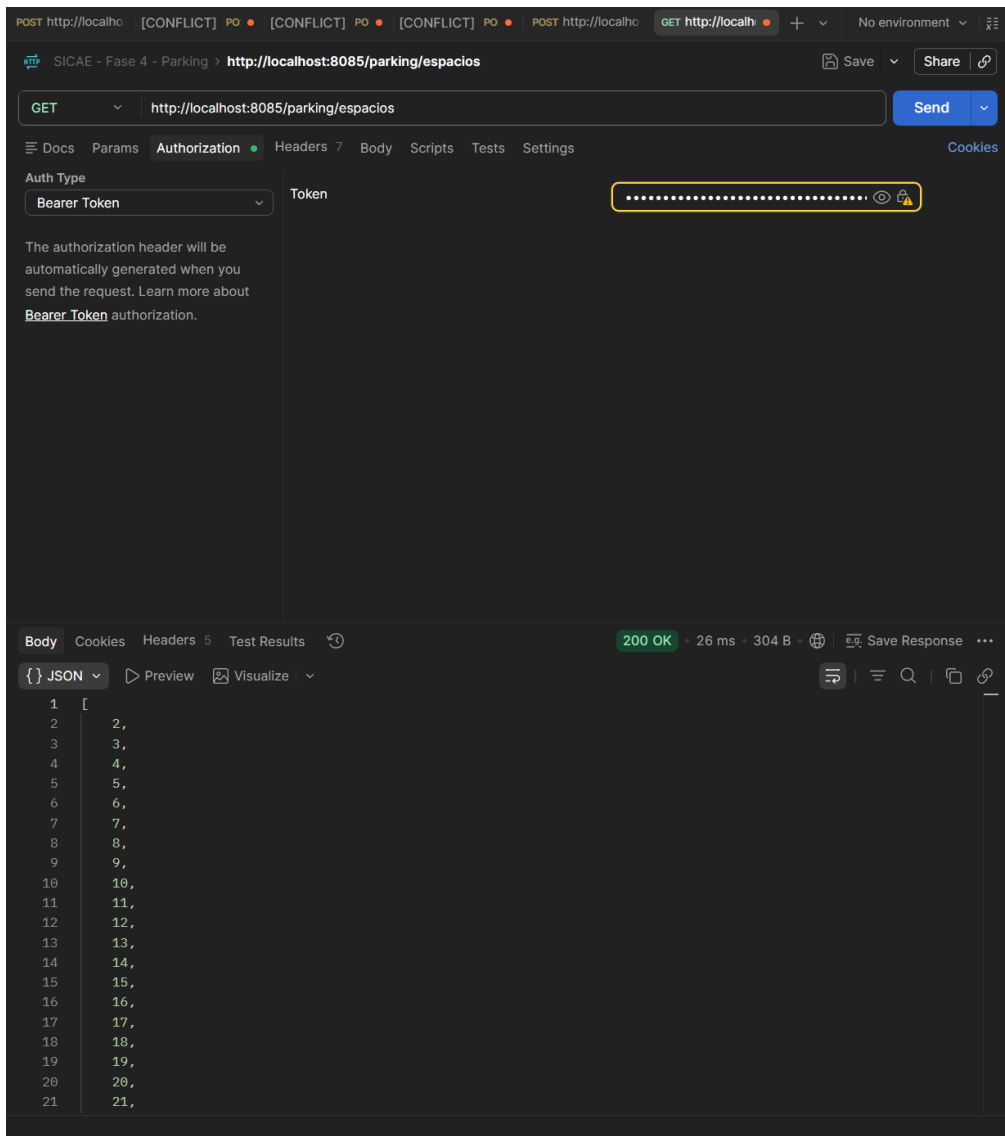


Ilustración 59 Evidencia de parkingService numero 2



Salida de auto del estacionamiento:

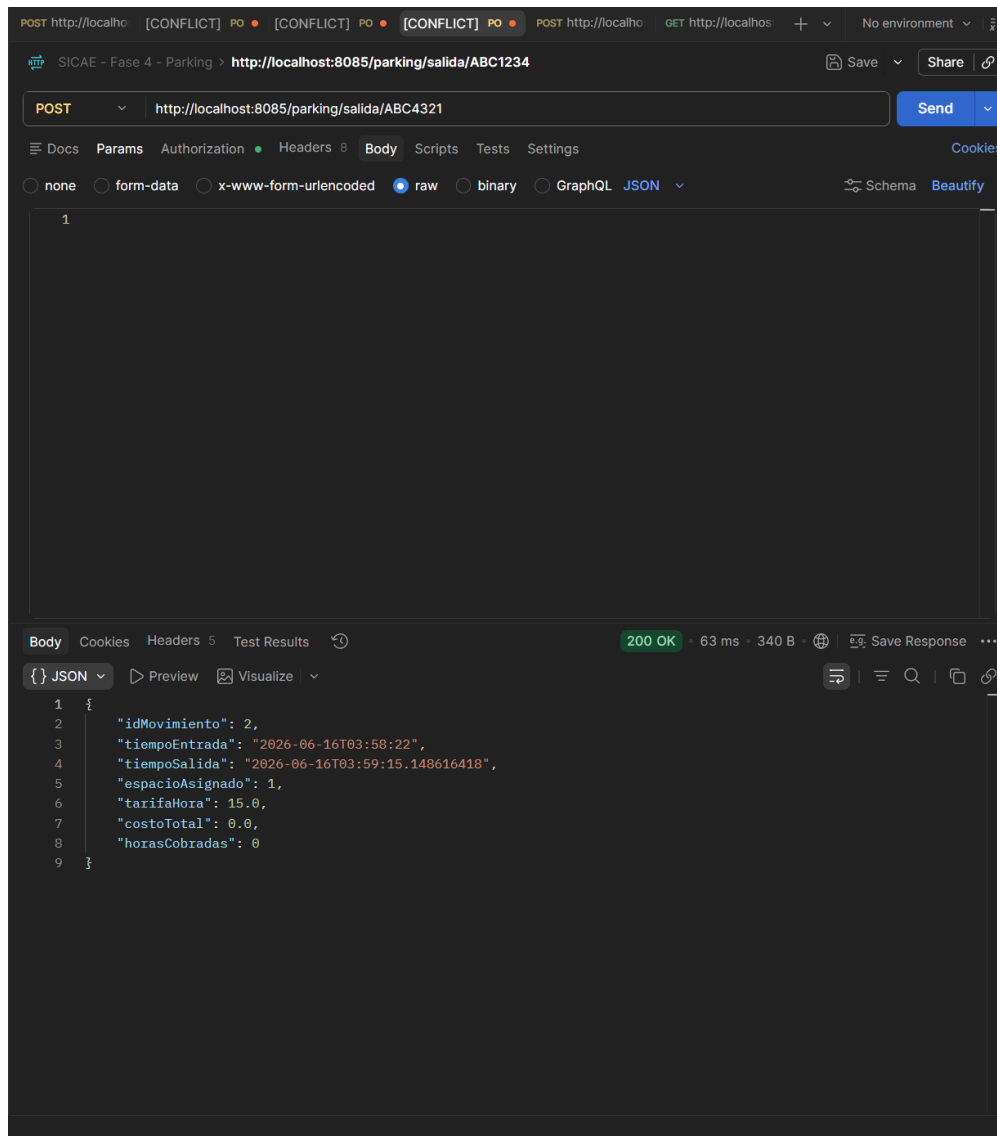


Ilustración 60 Evidencia de parkingService numero 3



Ver los cajones disponibles después de salida de estacionamiento:

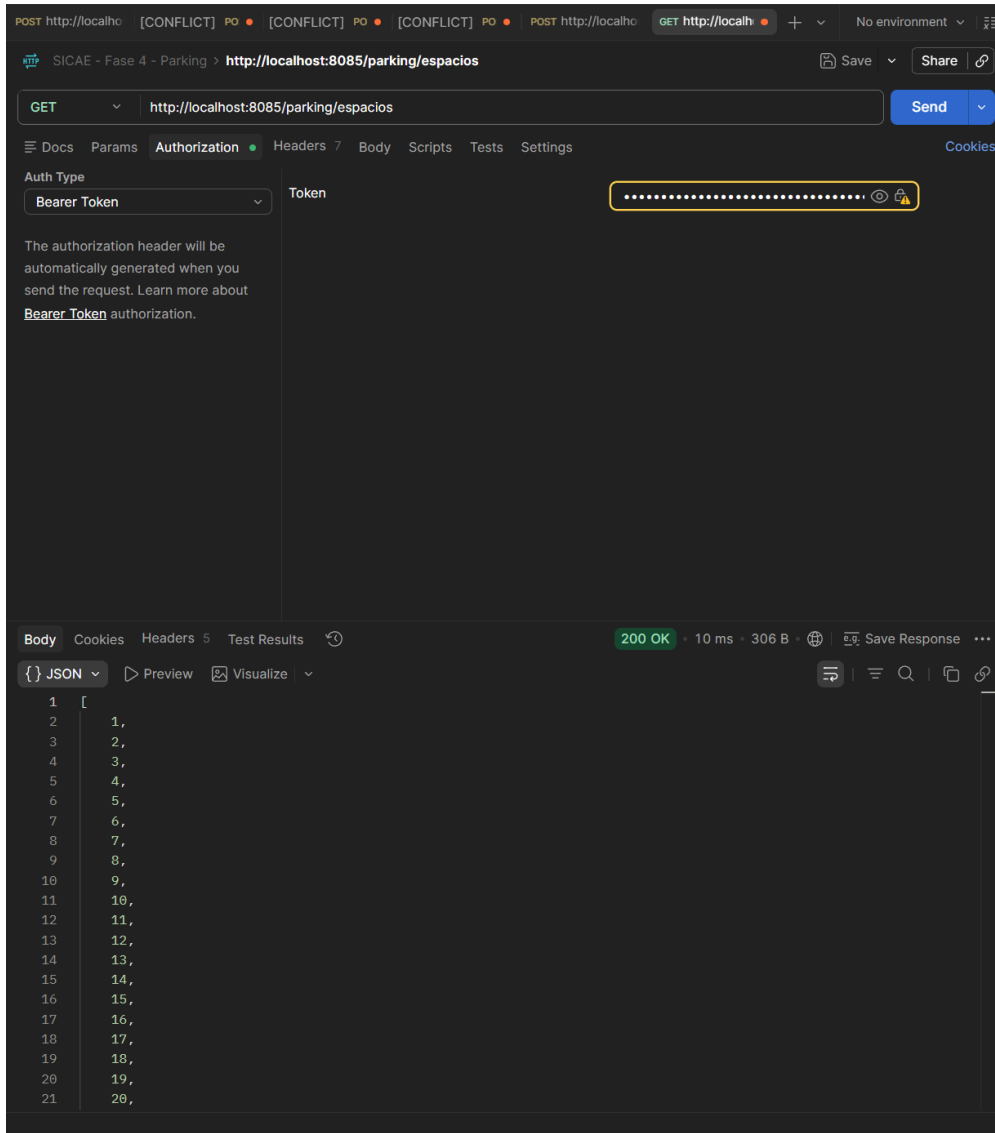


Ilustración 61 Evidencia de parkingService numero 4



Registrar Vehículo ya estacionado:

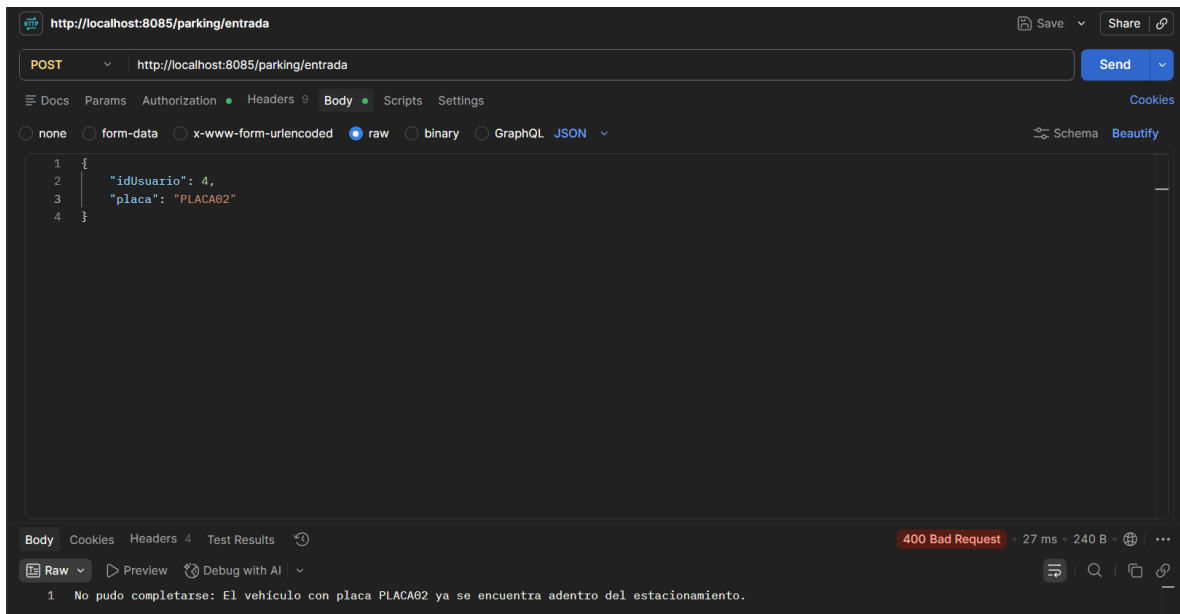


Ilustración 62 Evidencia de parkingService numero 5

Token inválido para ver espacios disponibles en estacionamiento:

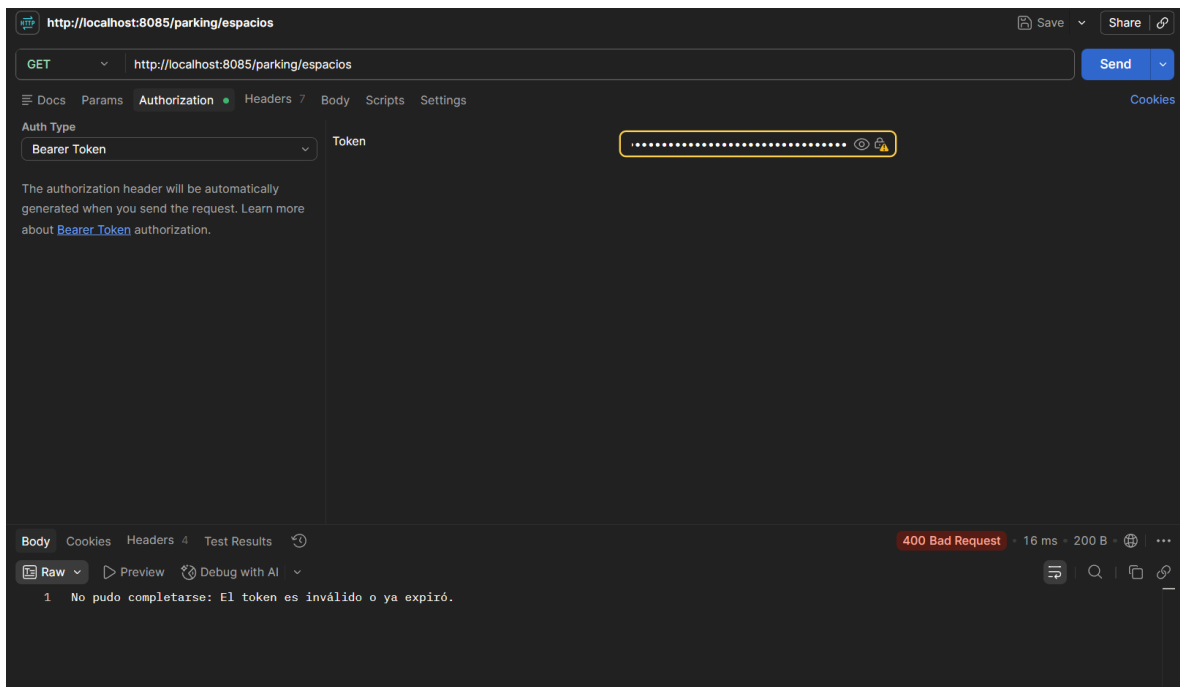


Ilustración 63 Evidencia de parkingService numero 6



Pasar el límite de registro de vehículos por usuario:

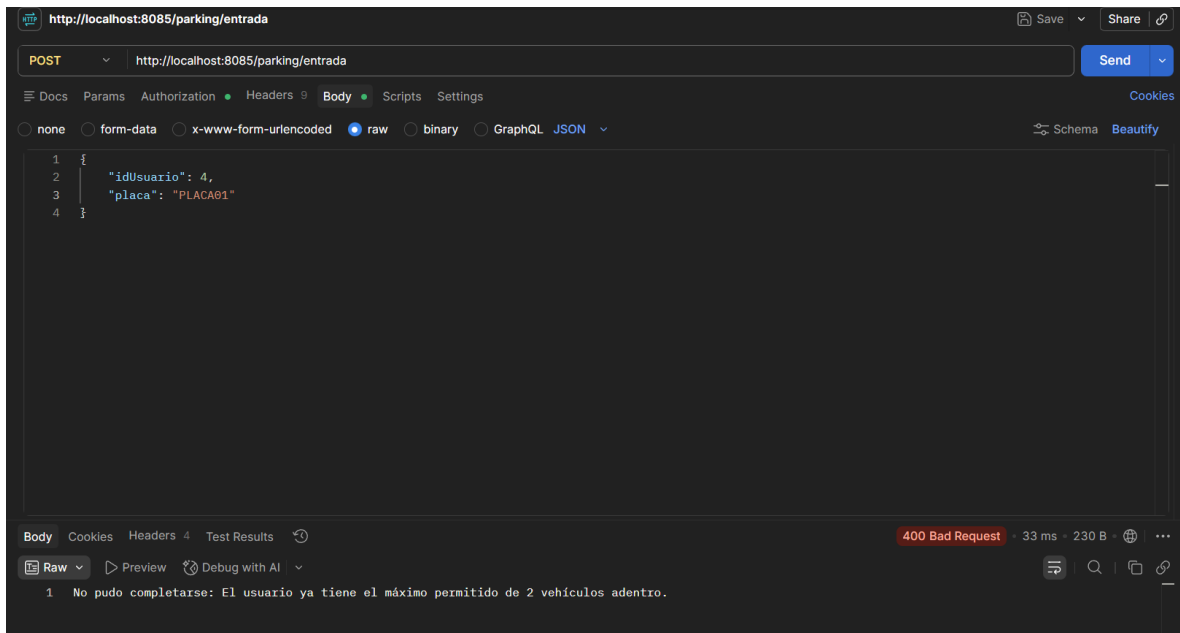


Ilustración 64 Evidencia de parkingService numero 7

El vehículo le pertenece a otro usuario:

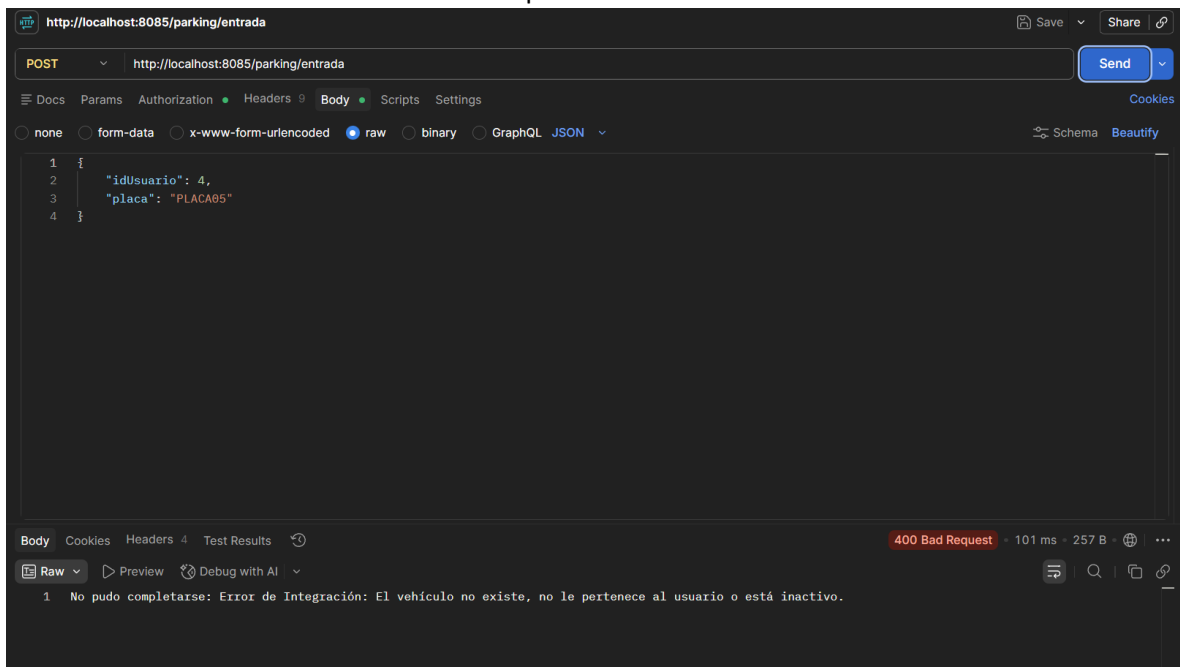


Ilustración 65 Evidencia de parkingService numero 8



Salida de vehículo inexistente en el estacionamiento:

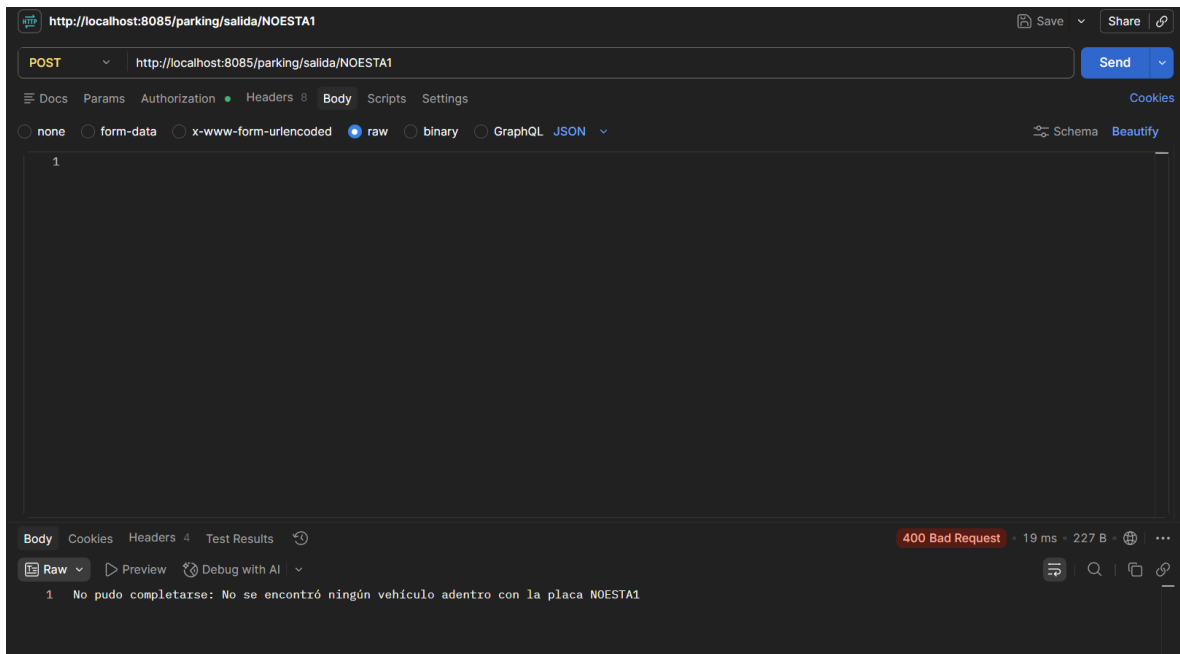


Ilustración 66 Evidencia de parkingService numero 9

Registrar la entrada de un auto cuando ya está lleno:

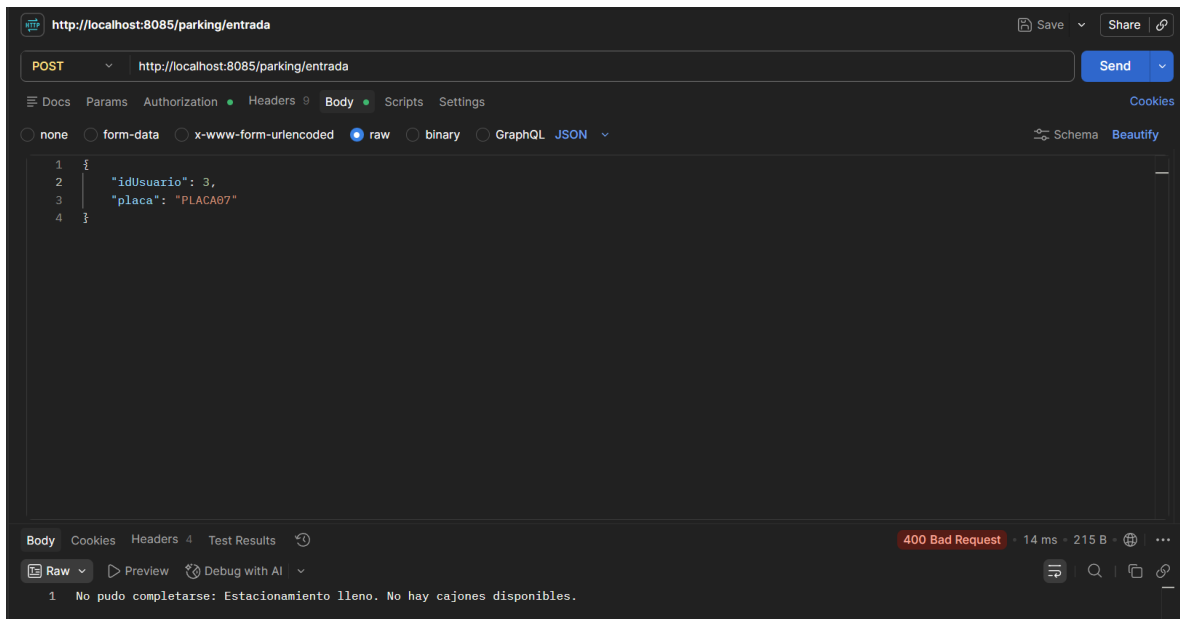


Ilustración 67 Evidencia de parkingService numero 10



Conclusiones

Resultados obtenidos

Como resultado del desarrollo e implementación del Sistema de Control de Acceso a Estacionamiento (SICAE), logramos construir y desplegar de manera exitosa una plataforma que cumple con la funcionalidad correspondiente basada en la arquitectura de microservicios orientada al dominio. El sistema cumple con los requerimientos técnicos y de negocio establecidos de manera satisfactoria, además esto garantiza la independencia, escalabilidad de cada uno de sus componentes.

Entre los principales resultados en el proyecto destacan los siguientes:

- **Arquitectura Distribuida y en Capas:** Desarrollamos cuatro microservicios independientes (AuthService, UserService, VehicleService y ParkingService). Cada uno lo estructuramos utilizando el patrón de diseño Controlador-Servicio-Repositorio, y así logramos una separación de responsabilidades.
- **Persistencia Desacoplada:** Cumplimos con el principio de base de datos por dominio, implementando los motores de bases de datos (utilizando PostgreSQL para usuarios y MySQL para los servicios de vehículos y estacionamiento). Ningún microservicio comparte base de datos con otro, evitando dependencias directas.
- **Seguridad Centralizada y Validación Descentralizada:** Se implementó un mecanismo de seguridad utilizando JSON Web Tokens (JWT) y cifrado de contraseñas con BCrypt. Esto quiere decir que el sistema permite que el servicio de autenticación genere el token, mientras que los demás microservicios pueden validar matemáticamente su autenticidad de forma local. Como beneficio optimiza los tiempos de respuesta.
- **Integración entre Servicios:** Logramos que la comunicación entre los microservicios fuera fluida y segura entre los microservicios utilizando peticiones HTTP REST mediante RestTemplate, esto permite que el sistema logre orquestar operaciones complejas validando información en tiempo real y a través de la red.
- **Despliegue Contenerizado:** Todo el ecosistema fue empaquetado y desplegado utilizando Docker y Docker Compose de manera exitosa. Además se configuraron tres servidores virtualizados con redes aisladas, asegurando que la solución sea portable y fácil de replicar en cualquier entorno.
- **Validación de Calidad:** Mediante el uso de herramientas principalmente Postman, se ejecutaron pruebas de todos los endpoints, comprobando que las reglas de negocio, la sanitización de datos y el manejo de errores (códigos de estado HTTP) funcionan con excelencia mostrando que funciona frente a escenarios reales.



Aprendizajes

Durante la realización de este proyecto aprendimos conceptos y habilidades fundamentales para nuestro perfil como ingenieros. En primer lugar, la elaboración de diagramas para representar toda nuestra arquitectura, cómo llegan, se tratan y envían tanto las peticiones como las respuestas en los servicios, y también saber representarla de forma gráfica. También aprendimos cómo es que la estructura de esta arquitectura facilita muchísimo tanto la codificación de las funciones que queremos realizar como la noción de cómo queremos que haya un flujo coherente para cada aspecto del negocio. A nivel de código, aprendimos aspectos fundamentales y rememoramos algunos otros, como el cómo solicitar una autenticación en una petición, cómo se trata dentro de nuestro servicio, aspectos como encriptar una contraseña con la herramienta Bcrypt y cómo pasamos de nuestro servicio a su base de datos.

En general, esto fue un vaivén tanto de aprendizajes viejos como nuevos. Tener el pensamiento de que sí hay cosas que sabemos, pero que podemos seguir aprendiendo e investigando, nos deja un grato sabor de boca.



Problemas encontrados

Durante el desarrollo de este proyecto, aun siendo un sistema, nos encontramos con dificultades individuales que cada uno tuvo que resolver. En primera instancia, durante el desarrollo de AuthService se presentaron dificultades, ya que no se sabía cómo utilizar Bcrypt y tampoco cómo hacer pruebas para chequear que las contraseñas fueran correctas. Al no saber, se pensó que estas podían ser desencriptables, pero resultó ser que no era posible, ya que solo se podían comparar hashes.

Durante el desarrollo de UserService hubo varios inconvenientes. Primero, importar y hacer funcionar todo lo que el integrante 1 del proyecto realizó. Una vez que se logró, otra de las eventualidades fue cómo comprobar la validez del token que nos dan en AuthService desde el login. Aquí hicimos uso de YouTube y páginas como Paradigma para tomar de base la información y transformarla a nuestra versión.

Para el desarrollo de VehicleService, el script original de la base de datos, en la tabla vehículos, no tenía la columna estatus y el requerimiento pedía activar/inactivar vehículos. Entonces, tuve que agregar un `estatus bit(1) NOT NULL DEFAULT b'1'` para que tuviera el atributo estatus y se pusiera por default en activado cuando se agregara uno nuevo. También actualicé la vista `vehiculofullinfo` y mi proyecto estaba compilando con Java 8, pero Spring Boot necesitaba Java 17, así que tuve que configurar el JDK.

Finalmente, para el desarrollo de ParkingService hubo errores con los puertos, donde se marcaba que el 8083 o el 8085 ya estaban ocupados. Vimos que era porque teníamos tanto NetBeans como el contenedor activo, entonces entraban en conflicto.

Para finalizar, el único problema que tuvimos al contenerizar los servicios y desplegarlos fue que AuthService, por accidente, lo cambiamos al puerto 8082. Ese estaba destinado a UserService y entraba en conflicto al desplegarse porque ambos intentaban ocuparlo.